Monthly Report for Modular Programming: Month 2

1- dars: Multiclass Classification

Binary Classification:

I. 2 ta classli **target** qiymat mavjud bo'lgan dataset kerak bo'ladi.
II. Confusion Matrix tushunish oson (0/1)
III. 0/1 ligi uchun modelni tushunish oson
IV. Faqat bitta classifier modelni talab qiladi.
V. One vs another (0/1, yes/no)

Multi-class classification:
I. Logistic Regression (orqasida **Sigmoid** funtion or Logistic function ([0;1]) oralig'ida ---> Binary Classification, Decision-Tree va Random Forest --> Binary va Multiclass)
II. Xaqiqiy xayotda ko'plab xolatlar multiclass: Masalan kodrdinata o'qidagi 3 xil classlar; snakes, dogs, and cats.
III. Data katta va balansli bo'lishi kerak.
IV. Yechim: Multi-classlarni ko'plab binary xolatlarga bo'lib olish.

2ta asosiy strategiya mavjud:
1. One-vs-Rest (OvR):
ayiq vs (odam+ asal),
asal+ (odam+ ayiq),
odam+ (ayiq +asal)

2. One-vs-One (OvO) --> Bu usul yordamida har bir class bosha har bir class bilan alohida model orqali solishtiriladi.

cat vs dog,
cat vs snake,
snake vs dog

OneVsRestClassifier:
-- Biz bir classification modelni xar bir classga uchun **train** qilamiz va bu yerda u bir classni qolganlarini yig'ilganalridan (**rest**) farqini o'rganadi.

Misol: 3 ta class

-Ayiq
-Odam
-Asal

Model 1: Ayiq vs (Odam + Asal)
Model 2: Odam vs (Ayiq + Asal)
Model 3: Asal vs (Ayiq + Odam)

Agar classimiz N ta bo'lsa N marta train qilamiz. Bu yerda 3 ta modellarni train qildik (1ta algoritmda).

OvR -->
Model 1: Ayiq vs (Odam + Asal) --> 0.65

Model 2: Odam vs (Ayiq + Asal) --> 0.18 ---- } Predicted class: Ayiq
Model 3: Asal vs (Ayiq + Odam) --> 0.25
OvR example 1:

```
from sklearn.multiclass import OneVsRestClassifier
                                .
                                .
                                .
ovr_model = OneVsRestClassifier(LogisticRegression(max_iter=1000, random_state=42))
ovr_model.fit(X_train, y_train
```

OvR (advantages):
1) Sodda va oson ishlatish
2) Agar classlar ozroq bo'lsa tez ishalydi.

OvR (disadvantages):
1) Imbalanced datasetlare bilan yaxshi ishlamaydi.

OvO: OneVsOneClassifier

Bu yerda biz classification modellarni xar bir class uchun **train qilamiz** va bu yerda xar bir classlarni qolganlari bilan yakkama yakka farqini o'rganamiz.
Misol uchun : 3 ta class
-ayiq
-odam
-asal
ayiq vs. odam
ayiq vs. asal
odam vs. asal

OvO example :

```
from sklearn.multiclass import OneVsOneClassifier
                                .
                                .
                                .
ovo_model = OneVsOneClassifier(LogisticRegression(max_iter = 1000, random_state=42))
ovo_model.fit(X_train, y_train)

y_pred_ovo = ovo_model.predict(X_test)
```

OvO formulasi: n(n-1)/2
OvO vs OvR larning asosiy farqi:
Modellar sonida va sekin ishlashida.

OvO:
Prediction
Model 1: **ayiq vs. odam (ayiq)**
Model 2: **ayiq vs. asal (ayiq)**
Model 3: **odam vs. asal (odam)**

----} Predicted class: Ayiq. Agar teng bo'lib qolsa confidence scorega ko'ra eng yuqori score olgnai predict qilinadi. Confidence --> Extimollar nazariyasi.
Confidence score formulasi:

$$confidence\ score = \frac{1}{1 + e^{-z}}$$

OvO (advantages):
1) Imbalanced datasetlarda ishaltish yaxshi
2) Medium classlarda xam yaxshi

OvO (disadvantages)
1) 10 ta class -> 45 models
2) 100 ta classda esa 4950 ta models
3) Nimaga olib keladi?:
    3.1 Ortiqcha chiqim, sarfga olib keladi.
    3.2 Dataset ko'p bo'lsa ko'p vaqt ketadi.
    3.3 Modelni sekin ishlashiga ham olib keladi.

Bu shunchaki biz train qilayotkanimizda, lekin hali model tayyorlab olgandan keyin, uni xaqiqiy ishga tushirayotgan paytimizda ham buni manitoring qilish, buni natijasini olishimiz ozgina qiyinchilik tug'diradi.

Lekin baribir aytib o'tilganidek bizni MLda algoritm ko'ramizmi biron bir usulmi, methodmi, qaysi birida qiymat yaxshi chiqishi uchun albatta ko'proq vaqt talab qiladi.

Multiclass classification umumlashgan holda:
1. Ko'plab class outputga ega bo'lgan dataset kerak bo'ladi.
2. Multi-class model:
    2.1 One-vs-Rest (OvR)
    2.2 One-vs-One (OvO)

Max iterations (parameter):

-Manzilimziga qadamba-qadam borish uchun bir qadam --> best solution
-Bazida oz bazida ko'p qadam kerak bo'ladi.
-Algorithms: 3 ta xolatda max_iter parameterinin ishlatib ketsak bo'ladi.
    --Linear (MLda)
    -- DL (Deep Learningda)
    -- Clustering (Unsupervised MLda)

Eng to'g'ri tushuncha:
I. Iteration = modelning weightlarni yaxshiroq tomonga bir marta yangilaydi.
II. Optimization = eng yaxshi weightlarni topish jarayoni max_iterning shu jarayonda modelga berilgan maksimal urinishlar soni.
III. LogisitcRegression (max_iter = 1000)
    -Model, eng yaxshi weightlarni topish uchun 1000 martagacha urinishi mumkin.
    - Agar max_iterni bermasak avtomatik (default = 100) olib ketadi.

2- dars: Class Imbalanced problem, Oversampling va Undesampling

Class imbalanced problem nima?
Hozirgi vaqtda problem real-world datasetlar uchun tabiy muammo.

Bunga misol masalan Heart Disease Predictionda 9% postive cases, 92 % esa negative cases.

Agar model shunday imbalanced datasetda train qilinsa nima bo'ladi?
1. Kichik miqdordagi classlarni ignore qiladi .
2. Kichik miqdordagi classlar ustida past natija beradi.
3. Umumiy natija baland bo'lishi mumkin lekin **biased performance** bo'ladi va bu **overfittingga** sabab bo'ladi.

Oversampling vs Undersampling
-- Sampling texnikasi:
    -- Oversampling
    -- Undersampling

-- Oversampling ---
Kichik miqdordagi classlarni qiymatlarini oshirish va katta miqdordagi classlar soniga yetkazish.

Oversampling techniqeues:
I. Random Oversampling -> Randomly dublicate qialdi. (kam classlarni)
    Masalan; class 1 ga -> 90 va class 0 ga-> 10 ta. Shu classni 0 ni class 1 ga tenglashtirib oladi.

ii. SMOTE (Synthetic Minority Oversampling Technique) -> New = A + α(B + A), 0<α<1
iii. ADASYN (Adaptive Synthetic Sampling)

Oversampling kichik miqdordagi classlarni qiymatlarini duplicate qiladi yoki yangi sintetik data pointlarni yaratadi.

Masalan: A = [10, 20], B = [20, 40] α = 0.5
New = [10, 20] + 0.5([10, 40] - [10, 20]), New = [15, 30] ==> Demak [15, 30] - yangi synthetic sample.

ADASYN (Adaptive Synthetic Sampling) : Modelga qiyin bo'layotgan minority samplelar atrofida ko'proq data yaratadi.

Ya'ni qaysi minority nuqtalar atrofida majority juda ko'p? Demak shu joy model uchuun qiiyin shu yerga ko'proq synthetic data yaratishim kerak.

Xayotiy analog: Sizda 2 ta guruh bor jami 100 ta talaba:
10 tasi yaxshi tayorlanmagan talabalr,
90 tasi yaxshi tayyorlangan talabalar
Modelga bu 2 guruhni ajratishni o'rgatmoqchsiz. Muammo yaxshi bilmaydigan uruh soni juda kam.
ADASYN nima qiladi? -> Avval yaxshi bilamydigan guruhdagi har bir odamga qaraydi va uning atrofida odamlar kimligiga qaraydi.

Masalan: Ali atrofida asosan bilimsiz talabalar -> Demak, Ali joylashgan hudud oson. Vali joylashgan hudduda esa ko'pchlik bilmli talabalar yig'ilgan - >Demak Vali joylashgan hudud qiyin. Model Vali kabi na'munalarni ajratish qiynaladi.

ADASYN shunda nima deydi?

Ali --> bilimsiz studenlar atrofida ozgina yangi data.
Vali --> biliimsiz studentlarga ko'proq yangi data

yani ADASYN hamma joyga bir xil iqdorda data qo'shmaydi.
Qiyin joy --> ko'proq data
Oson joy --> kamroq data

Formulasi:

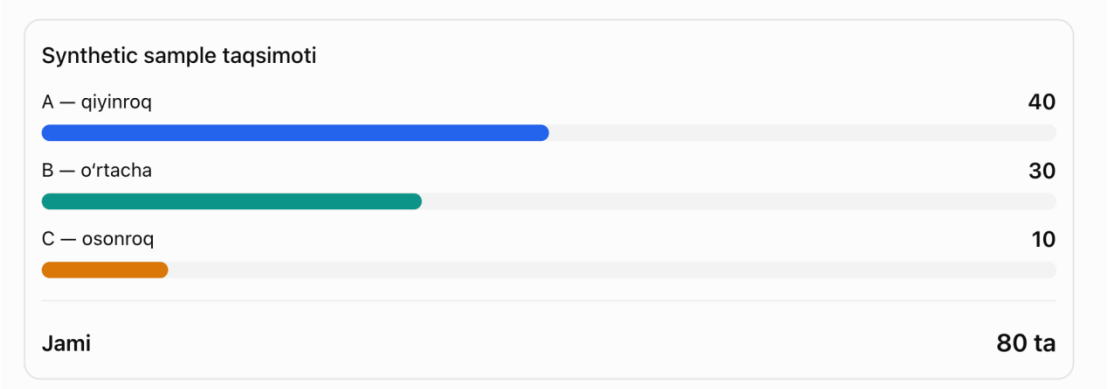
$$\underbrace{r_i = \frac{\Delta_i}{k}}_{\text{Qiyinlik darajasi}} \qquad \underbrace{g_i = \frac{r_i}{\sum r_j} G}_{\text{Synthetic sample soni}}$$

1. Qiyinlik darajasi
2. Qancha synthetic data yaratishni hisbolash

Faraz qilaylik, 3 ta minortiy sample bor:

A qiyinlik darajasi r_i = 0.8
B qiyinlik darajasi r_i = 0.6
C qiyinlik darajasi r_i = 0.2
Jami 80 ta synthetic sample yaratmqochimiz.

1-qadam: $\sum ri$ = 0.8 + 0.6 + 0.2 = 1.6
2-Har bir sample uchun hisoblaymiz:
g_A = 0.8 / 1.6 * 80 = 40
g_B = 0.6 / 1.6 * 80 = 30
g_C = 0.2 / 1.6 * 80 = 10



Shunda ADASYN algoritmi qiyinroq minority sample atrofida ko'proq synthetic data yaratadi.

- A qiyinroq -> 40 ta
- B o’rtacha -> 30 ta
- C osonroq -> 10 ta

Coding xolatda:

```
from collections import Counter  # sanab berish uchun
from imblearn.over_sampling import RandomOverSampler, SMOTE, ADASYN
                                        .
x = np.array([
[1], [2], [3]
])
y = np.array([
[4], [5], [7], [8], [9], [10], [11]
])
                                        .
                                        .
ros = RandomOverSampler(random_state=42)
X_ros, y_ros = ros.fit_resample(x, y)
print(“\n Random Oversampling class distributions” , Counter(y_ros))
print(“Random Oversampling samples: \n”, X_ros.flatten()
```

SMOTE

```
smote = SMOTE(random_state=42, k_neighbors = 1) , # _neighbors - katta datasetlarda shart emas olish

X_smote, y_smote = smote.fit_resample(x, y)
print(“\n SMOTE class distribution: “, Counter(y_smote))
print(“SMOTE samples: \n”, X_smote.flatten())
```

ADASYN

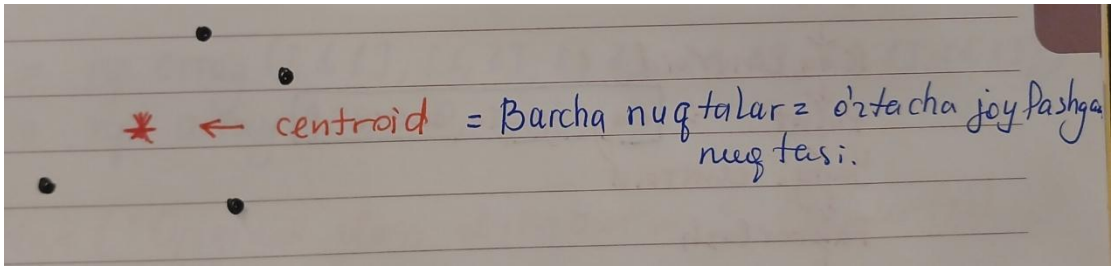
```
adasyn =  ADASYN(random_state=42, n_neighbors = 1)
X_ada, y_ada = adasyn.fit_resample(x, y)

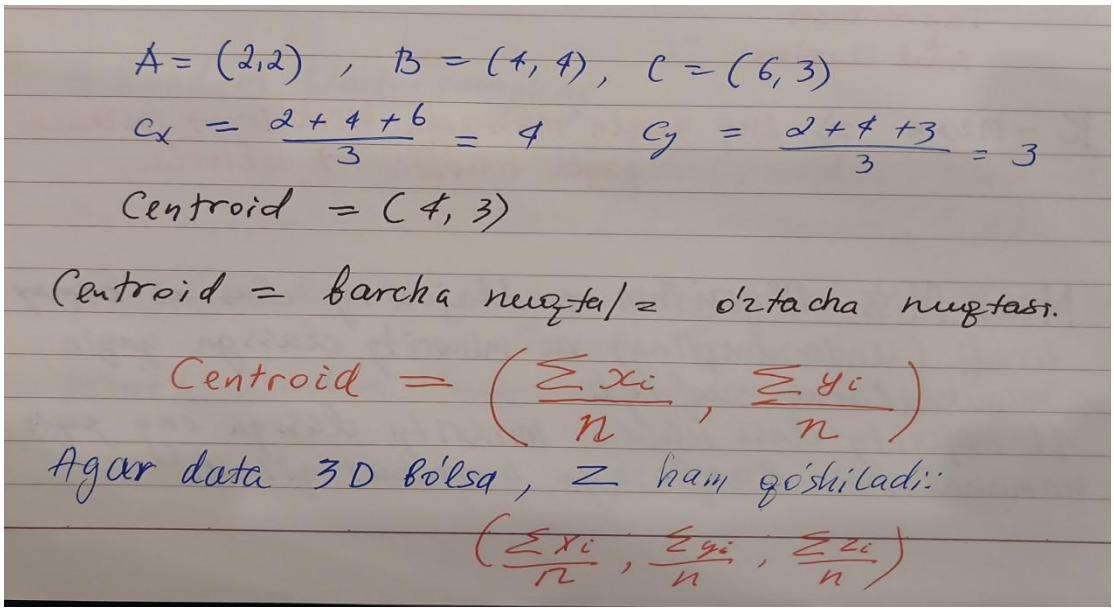
print(“\n ADASYN class distribtution: “, Counter(y_adasyn))
print(“ADASYN samples: \n”, X_adasyn.flatten())
```

Undersampling:

Majority classdagi qiymatlarni kamaytirish. Asosiy texnikalar:
1. Random Undersampling
2. Cluster Centroids
3. Near Miss Algorithms

Centroid nima?
Tasavvur qil: Bir xonada 5 ta odam turibdi. Ularning o’rtasida bitta joy bor. O’sha joy - Centroid deyiladi.
Datada ham huddi shunday:





K-means -datalarni o'xshashligiga qarab k ta guruhga (cluster) ajratiladigan algorithm. Masalan, k = 3 desak, 3 ta cluster topadi.

Jarayon:
1. k ta centroid tanlaydi.
2. Har bir data pointni eng yaqin centroidga biriktiradi.
3. Har bir cluster uchun yangi centroidni (o'rtacha nuqtani) hisbolaydi.
4. Centroidlar o'zgarmaguncha 2-3 qadamlarni takrorlaydi.

Data -> k ta centroid -> point (eng yaqin centroid) -> yangi centroid -> takrorlanish -> final clusters

L-means -> eng yaqin markazga biriktir -> markazni qayta hisobla -> takrorla

Near Miss Algorithm -> Majority classni kamaytiradi (undersampling) va minority classga yaqin namunalarni saqlaydi. U majority class ichida minority class eng yaqin namunalarni saqlashga harakat qiladi.

Coding:

```
from collections import Counter  # sanab berish uchun
from imblearn.under_sampling import RandomUnderSampler, ClusterCentroids, NearMiss

x = np.array([
```

```
[1,1], [2,1], [3,1], [4,1], [5,1]
])
y = np.array([
[4, 5, 6, 7, 8, 9, 10, 11]
])
                                        .
                                        .
ros = RandomUnderSampler(random_state=42)
X_rus, y_rus = rus.fit_resample(x, y)
print("\n Random Undersampling class distributions" , Counter(y_ros))
print("Random Undersampling samples: \n", X_ros.flatten())


# Cluster centorids
cc = ClusterCentroids(random_state=42)
X_cc, y_cc = cc.fit_resample(x, y)

print("\n Cluster Centroids class distributions" , Counter(y_cc))
print("Cluster Centroidssamples: \n", X_cc))

# Nearmiss Algorithms
nm = NearMiss(version = 1, n_neighbors = 1)
X_nm, y_nm = nm.fit_resample(x, y)
print("\n Near Miss class distributions" , Counter(y_nm))
print("Near Miss samples: \n", X_nm))
```

Oversampling advantages:

I. Datalarni originalligini saqlab qoladi.
II. Minority classlarni ulushini oshiradi -> Minority classlarning valuelarini predict bo'lishi imkoniyatini paydo qiladi.
III. Yetarli computing resourcelar bilan yaxshi ishlaydi.

Oversampling disadvantages:

Overfittingga sabab bo'ladi (Random Oversampling)
Training time oshib ketadi.
Undersampling advantages:
I. Training tezroq bo'ladi
II. Sodda va effective
Undersampling disadvantages:
I. Muhim bo'lgan datalarni yo'qolishiga sabab bo'ladi.
II. Underfittinga sabab bo'lishi mumkin.

4-dars: Hyperparameters

i. Eng zo'r algoritmlar ham yomon natija chiqarishi mumkin (overfitting, underfitting).
ii. Hyperparameterlar bu algoritmni settings (sozlamalari) xisoblanadi.
iii. Noto'g'ri Hyperparameter (settings) --overfitting va underfittingga olib keladi.

Decision Treeda qanday Hyperparameterni bilasi? --->max_depth =1 ->underfitting, max_depth=100->overfitting.

Linear Regression	fit_intercept, normalize
Logistic Regression	C, penalty
Decision Tree	max_depth, min_samples_split
Random Forest	n_estimators, max_features
Ridge Regression	alpha, solver
Lasso Regression	alpha, max_iter

Sklearnda hamma parameterlar nima vazifa bajarishini ko'ramiz va shpalgalka qilib olamiz o'zimiz uchun.

Hyperparameter vs Parameter

Hyperparameter:

- Configuration o'zgaruvchilar (o'zimiz sozlamalarni beradigan o'zgaruvchi hisoblanadi), berilayotgan algoritmning ishlashi, qayergacha ishlashi, nimalardan tashkil topishi, o'shani o'rganuvchi sifatida bevosita biz beramiz. Ya'ni control bizning qo'limizda bo'ladi.

- Trainingdan oldin qo'yiladi. Trainingning protsesini nazorat qiladi (Trainingdan oldin qo'yiladi.--> max_depth =-10 va max_depth =50 --> 1. vaqtga tasir qiladi, 2. modelni qayerlargacah o'rganishini taminlaydi.
- Ular datralardan o'rganmaydi (ammo dataga tasir qiladi).

Parameterlar:

- Internal coefficient (weights, ammo parameterlar datalardan o'rganadi).
- Datalardan automatic o'rganadi va manually kerak emas. Examaple: w, b (weights, bias) y = wx + b

Hayotiy analog: gazni qanchalik bosish, tezlanish qanday bo'lish kerak.

Hyperparamters: 1. Gear mode (auto, manual), 2. Cruise control speed (o'zimiz speedni belgilashimiz mumkin).

Hypeparameter Tuning

Model uchun optimal bo'lgan (eng zo'r bo'lgan) hyperparmerterlarni izlash processi.
Nima uchun muhim?

- Overfitting bilan underfittingni oldini oladi.
- Model aniqlik va chidamlilikni yaxshilaydi.
- Reduce bias-variance trade-off
- Saves time and computational resources.

Nima olib keladi?

- Yuqori aniqlik
- Kam error
- Best Generalization

Ularning turlari:
1. MANUAL SEARCH:
Foydaliligi: Kichik va sodda modellarda past.
Qachon ishlatish: Tez tajribalar, kam parameterli oddiy modellar.

Misol: max_depth = 3 yoki max_depth =5 qiymatini kichik Decision Tree uchun sinash.
Advanbtage: Oddiy hech qanday kutubxona talab qilinmaydi.
Disadvantage: Samarasiz, eng yaxshi qiymatni o'tkazib yuborish mumkin.

2. GRID SEARCH: Grid Search Hyperparameter Tuninging eng mashxur vakillaridan biri xisoblanadi.
Benefits: Kichik ma'lumotlar va kam parameterlar uchun foydali.
When is it used: Kichik parameterlar soni bilan to'liq tekshirish
For example: Random forest uchun: n_estimators = [50,100] va max_depth = [3,5].

Advantages: Barcha kombinatsiyalar tekshiriladi.

3. RANDOM SEARCH: Random Search bu max_depth = [3, 5, 10] shular ichidan randomly oaldi.
Benefits: Katta parameterlar maydoni uchun yuqori.
Comparisons: Manualldanm yaxshiroq, ammo GridSearchdan yomonroq.
When is it used: Katta modellar (XGBoost, Deep Learning) bilan.
For example: n_estimators, max_depth, min_samples_spliting tasodifiy kombinatsiyalarni yaratish.
Advantages: Grid Searchdan tezroq, yaxshi natijalarni tez topadi.
Disadvantages: Eng optimal qiymatni o'tkazib yuborishi mumkin.

4. BAYESIAN OPTMIZATION:
Benefits: o'rta va katta modellar uchun juda yuqori.
When is it used: Har bir model muhim / qimmatli bo'lgan hollarda (ko'p daraxtli,. RF va Nueral Networks).
For example: XGBoost "learning_rate", va "n_estimators"ni optimallashtirish.

Comparisons: **Grid Search = Bayesian Optimization > Randomly Search and Manual Search.**

Advantages: Samarali, ehtimollik (probability) modellari yordamida optimumni topadi.
Disadvantages: Complex libraries talab qiladi (scikit-optimize, Optuna).

5. OPTUNA / Automated methods

Benefits: Katta va murakkab modellar uchun juda yuqori.
when is it used: Complex pipeline, deep learning.
For example: RF uchun n_estimators, max_depth,min_samples_split, min_samples_leaf ni birga tuning qilish.

Advantages: Kuchli, pruning (modeldagi keraskiz, yoki kam foyda beradigna qismlarni olib tashlash orqali modelni soddalashtirish usuli), multi-objective-optimization, distributed tuning imkoniyati.

Disadvangtages: Kutubxona o'rnatish va API ni o'rganish talab qilnadi.

Grid Search: Amaliy
```
from sklearn.model_selection import GridSearchCV

# Split dataset
X_train, X_test ..
```

```
# Define parameter grid
param_grid = {
        “n_estimator” : [50, 100, 150],
        “max_depth: : [2,4,6]
}

# Grid Search
grid_search = GridSearchCV(RandomForestRegressor(), param_grid, cv = 3) -->
```
cv=3 bu 3-fold cross validation. [DATASET] 3-qismga bo’linadi.

- FOLD 1
- FOLD 2
- FOLD 3

1. FOLD 2 + FOLD 3 -> Train validation : FOLD 1
2. Train: FOLD 1 + FOLD 3 validation: FOLD 2
3. Train : FOLD 1 + FOLD 2 valdiation : FOLD 3 --> keyin shu 3 tasini meanin olinadi.

```
grid_search.fit(X_train, y_train)
print(“Grid Search Best Params: “ , grid_srearch.best_params_)

# Train final model with the best hyperparameter
rf_grid = RandomForestRegressor(**grid_search.best_params_)
rf_grid.fit(X_train, y_train)

# Predictions and accuracy
y_pred_grid = rf_grid.predict(X_test)
r2_grid = r2_score(y_test, y_pred_grid)
print(“Grid Search Accuracy: “, r2_grid)
```

RANDOM SEARCH: Hammasini tekshirmayman, tasodifan bir nechta kombinatsiyani tekshiraman.

```
from sklearn.model_selection import RandomizedSearchCV
import numpy as np

# Train test split process core
…..

# Define parameter distribution
param_dist = {
        “n_estimator” : [50, 100, 150],
        “max_depth” : [2,4,6]
}
```

Bu kombinatsiya : 3*3 = 9
Lekin biz n_iter = 5 deb qo’yganmiz. Shuning uchun 9tasini hammasini emas faqat 5 tasini tasodifan kombinatsiya tanlanadi.

Masalan:
50 - 2
100 - 6
150 - 4

50 - 6
150 - 2
Boshqa 4 tasini umuman tekshirlmasligi mumkin.

sn_iter = 5, eng muhim joy -> Bu 5 ta parameter combination sinab ko'r degani.
Har bir cv =3, shuning uchun 5*3 = 15 ta training Grid Searchda esa 9*3 = 27 bo'ladi.
Demak Random Search tezroq.

```
random_search.fit(X_train, y_train)
print("Random Search pramas: ", random_search.best_params_)

# Train final model with best hyperparameters.
rf_random = RandomForestRegressor(**random_search_)
rf_random.fit(X_train, y_train)

# Predictions and accuracy

y_pred_random = rf_random.predict(X_test)
r2_random = r2_score(y_test, y_pred_random)
print("Random Search R2 score: ", r2_score)
```

Random search absolute besr qiymatnintopsihga kafolat yo'q.
lekin katta search spaceda ko'pincha juda samarali.

BAYESIAN OPTIMIZATION --> REALLY IMPORTANT HYPERPARAMETER TUNING TECHNIQUE

```
# USE PROBABILITY (pip install scikit-optimize)

from skopt import BayesSearchCV
from sklearn.model_selection import train_test_split
form sklearn.metrics import accuracy_score

# Fresh train/test split
..

# Define search space
param_space = {
        "n_estimators":(50, 150)  # integer rage
        "max_depth": (2,6)  # integer range
}

# Bayesian Search
bayes_search = BayesSearchCV(RandomForestRegressor(),
        param_space, n_iter=10, cv=3, random_state=42)
bayes_search.fit(X_train, y_train)
print("Bayesian Optimization Best Params: ", bayes_search.best_params_)



# Train final model with best hyperparameters
rf_bayes = RandomForestRegressor(**bayes_search.best_params_)
rf_bayes.fit(X_train, y_train)
```

Predictions and r2 score
y_pred_bayes = rf_bayes.predict(X_test)
r2_score_bayes = r2_score(y_test, y_pred_bayes)

Oldin sinov natijalaridan foydalanib, keyin qaysi hyperparameter promising ekanligni aniqlayman deydi.

Taxminiy jarayon --> :

trial 1
n_estimators = 70
max_depth = 3
r2_score = 0.60

trial 2
n_estimators = 120
max_depth = 3
r2_score = 0.73

Keyin model buni ko'rib:
Aha, katta n_estimators va max_depth yaxshi ko'rinayapti deb kehyinigi trailni shu atrofida tanlashi mumkin.

trial 3
n_estimators = 135
max_depth = 5
r2_score = 0.76

final trial 4
n_estimators = 145
max_depth = 5
r2_score = 0.76

yani:
TRIAL
↓
RESULT
↓
BETTER NEXT TRIAL
↓
RESULT
↓
LEARN
↓
…….. --> manashu "SEQUENTIAL OPTIMIZATION" deyiladi.

Bayesin Optimization ichki mantig'i

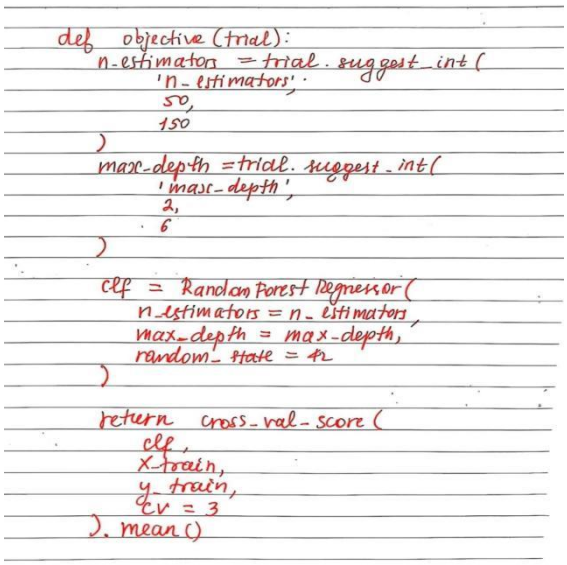
1. Surrogate Model -> Qayerlardan yaxshi natija chiqish mumkinligini taxmin qiladi.
2. Acquisition Function -> keyingi qaysi nuqtani sinash foydaliroq?

Bular qismlari va shu bilan birga strategiya ham mavjud
Exploration : Yangi hududlarini ko'rish -> "Bularni boshqa joyda juda yaxshi parameter bordir".
Exploration: Allaqachon yaxshi topilgan joy atrofiga qarash --> "shu hudud yaxshi ekan, yaqinlarini tekishirib ko'ray".

Bu juda muhim “concept”.

5.OPTUNA

Optuna ham aslida smart hyperparameter optimization framework.
Ularning ishlashi Bayesian Optimiationga yaqin. Lekin Optuna ancha flexible.



Maqsad: Optuna yordamida Random Forest modelining eng yaxshi hyperparameterlarini topish, Cross-Validation orqali baholash va yakuniy R^2 score hisoblash.

```
import optuna

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score
from sklearn.metrics import r2_score


# 1. Objective function
def objective(trial):

    n_estimators = trial.suggest_int("n_estimators", 50, 200)
    max_depth = trial.suggest_int("max_depth", 2, 15)

    model = RandomForestRegressor(
        n_estimators=n_estimators,
        max_depth=max_depth,
        random_state=42,
        n_jobs=-1
    )
```

```
    scores = cross_val_score(
        model,
        X_train,
        y_train,
        cv=3,
        scoring="r2"
    )

    return scores.mean()


# 2. Create Optuna study
study = optuna.create_study(direction="maximize")

# 3. Run optimization
study.optimize(objective, n_trials=10)

print("Best Parameters:", study.best_params)
print("Best CV R2:", study.best_value)


# 4. Train model with best hyperparameters
rf_optuna = RandomForestRegressor(
    **study.best_params,
    random_state=42,
    n_jobs=-1
)

rf_optuna.fit(X_train, y_train)


# 5. Predictions and final evaluation
y_pred_optuna = rf_optuna.predict(X_test)

r2_score_optuna = r2_score(y_test, y_pred_optuna)

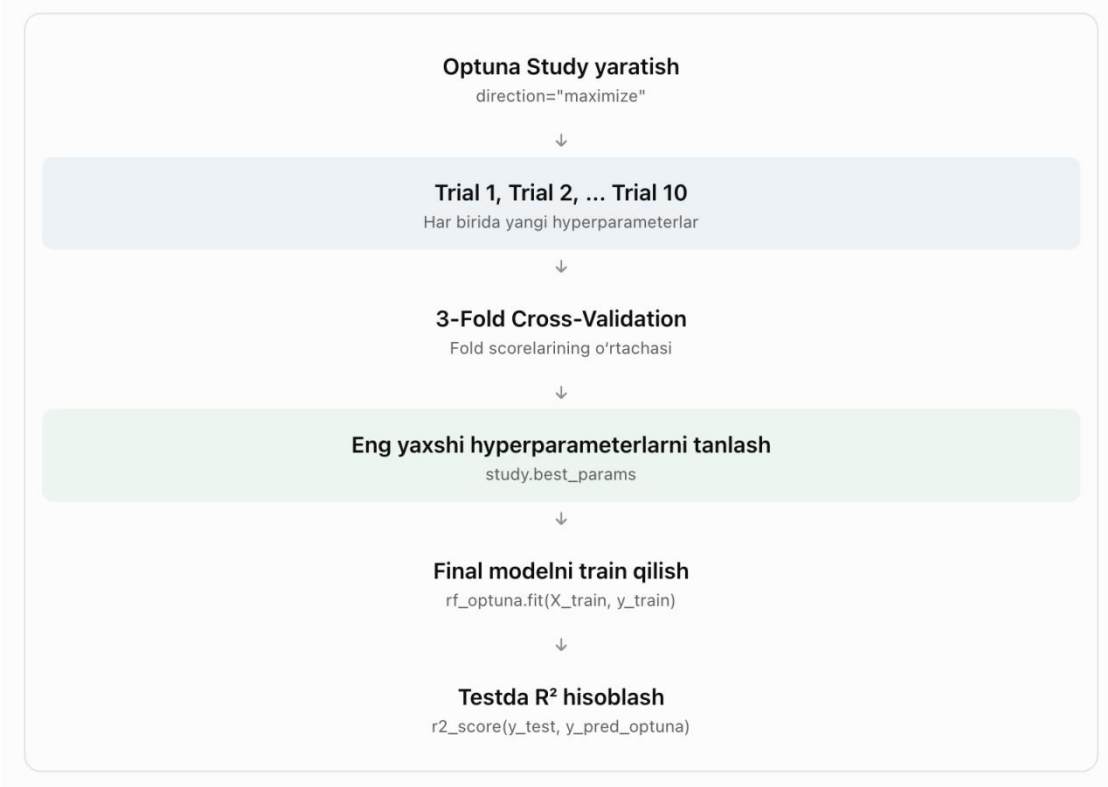
print("Optuna R2:", r2_score_optuna)
```

Tushuncha	Vazifasi
trial	Bitta hyperparameter tajribasi
suggest_int()	Parametr qiymatini tanlaydi
n_estimators	Random Forestdagi daraxtlar soni
max_depth	Daraxtning maksimal chuqurligi
objective()	Modelni baholab, CV score qaytaradi
cross_val_score()	Cross-Validation bajaradi
cv=3	3-fold Cross-Validation
scores.mean()	Uchta foldning o‘rtacha natijasi
create_study()	Optuna optimizatsiya jarayonini yaratadi
direction="maximize"	R^2 qiymatini maksimal qilish
n_trials=10	10 ta tajriba bajarish
study.best_params	Eng yaxshi hyperparameterlar

study.best_value	Eng yaxshi o‘rtacha CV R^2
****study.best_params**	Topilgan parametrlarni modelga uzatadi
fit()	Modelni o‘qitadi
predict()	Bashorat qiladi
r2_score()	Yakuniy R^2 natijasini hisoblaydi

Misol: Birinchi trialda n_estimators=72, max_depth=3 tanlandi. Cross-Validation natijalari 0.65, 0.68, 0.66 bo‘lsa, o‘rtacha $R^2 \approx 0.663$. Optuna boshqa triallar bilan solishtirib, eng yuqori o‘rtacha R^2 bergan parametrlarni tanlaydi.

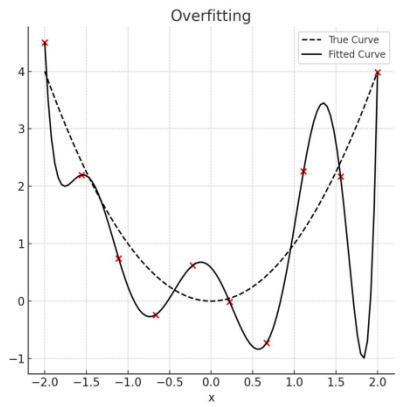
Muhim: R^2 uchun maximize, MSE/RMSE/MAE uchun minimize ishlatiladi. Test dataset hyperparameter tanlashda qatnashmasligi kerak. Preprocessing zarur bo‘lsa, uni Pipeline ichida bajarish lozim.



6-dars: Hyperparameters







Underfitting — model ma'lumotlardagi asosiy bog‘liqliklarni ham yetarlicha o‘rgana olmagan holat.

Good Fit / Generalization — model asosiy qonuniyatlarni o‘rganib, yangi ma'lumotlarda ham yaxshi natija ko‘rsatadigan holat.

Overfitting — model training ma'lumotlariga haddan tashqari moslashib, tasodifiy shovqin va keraksiz tafsilotlarni ham o‘rganib olgan holat.

asosiy fikr:

Overfitting — yodlab olish. Generalization — umumlashtirish.

2. Overfitting nima va uni qanday aniqlaymiz?

Model training datasetda juda yaxshi natija ko‘rsatadi, ammo yangi yoki ilgari ko‘rilmagan ma'lumotlarda yomonroq ishlaydi.

Asosiy belgilari:Training error juda past. Validation yoki test error ancha yuqori. Training va validation natijalari orasida katta farq mavjud.

Masalan:

Metric	**Training**	**Validation**
Accuracy	99%	72%
Error rate	1%	28%

Bu overfittingning kuchli belgisi bo‘lishi mumkin.

Overfittingning asosiy sabablari:

1. Modelning haddan tashqari murakkabligi: juda chuqur daraxtlar, ko‘p parametrlar yoki keragidan ortiq neyronlar
2. Training ma'lumotlarining yetarli emasligi yoki muhim holatlarni qamrab olmasligi.
3. Shovqinli, noto‘g‘ri belgilangan yoki sifatsiz ma'lumotlar.
4. Keraksiz featurelar va ortiqcha murakkablik.

Overfittingni kamaytirish usullari:

1. Model murakkabligini kamaytirish (reduce model complexity).
2. Ko‘proq va sifatli training data yig‘ish.
3. Feature Selection yoki Dimensionality Reduction.
4. Regularization: L1 yoki L2.
5. Neural Networklarda Dropout.

3. Underfitting nima va uni qanday aniqlaymiz?

Underfitting: model juda sodda bo‘lgani yoki yetarlicha o‘qitilmagani sababli ma'lumotlardagi muhim bog‘liqliklarni o‘rgana olmasligi.

Asosiy belgilari:

Training accuracy past.

Training error yuqori.

Validation/test error ham yuqori.

Masalan:

Metric	Training	Validation
Accuracy	61%	59%
Error rate	39%	41%

Model training ma'lumotlarida ham yaxshi ishlamayapti.

Underfittingni kamaytirish usullari:

1. Model murakkabligini oshirish (increase model complexity), masalan, qo‘shimcha qatlamlar yoki mosroq nonlinear model.
2. Ko‘proq foydali va relevant featurelardan foydalanish.
3. Haddan tashqari kuchli regularizationni kamaytirish.
4. Modelni uzoqroq o‘qitish yoki hyperparameterlarni optimallashtirish.

4.Uchta holatni qanday ajratamiz?

Holat	Training performance	Validation/Test performance
Underfitting	Past	Past
Overfitting	Juda yuqori	Ancha past
Good Fit	Yaxshi	Yaxshi va trainingga yaqin

1. Training ham, validation ham yomon → Underfitting ehtimoli.
2. Training juda yaxshi, validation yomon → Overfitting ehtimoli.
3. Ikkalasi ham yaxshi va farq kichik → Good Fit / yaxshi generalization belgisi.

5. Memorization va Understanding

Memorization	Understanding
Short-term	Long-term
Stored in working memory	Stored in long-term memory
Unconnected to real learning	Foundation for deeper learning
Information is available for quick recall	Information can be used in real-world applications
Limited to words and basic sentences	Extends to processes, systems and concepts
Literal interpretation only	Deeper and broader interpretation
Does not necessarily lead to insight or creativity	Helps connect ideas, reason and synthesize information

Agar biz buni Machine Learning bilan bog‘lasak, memorization training ma'lumotlaridagi alohida namunalarni eslab qolishga, generalization esa o‘rganilgan qonuniyatlarni yangi ma'lumotlarga qo‘llashga o‘xshaydi.

6. Parameters va Hyperparameters

Parameters are learned from the data.

Hyperparameters control how the model learns.

Parameters	Hyperparameters
Training davomida o‘rganiladi	Odatda trainingdan oldin belgilanadi yoki tuning orqali tanlanadi
Masalan, ANN weights va bias	Masalan, learning rate, batch size, max_depth
Model ma'lumotlardan moslashtiradi	Tadqiqotchi yoki optimization algoritmi tanlaydi

masalan:

```
RandomForestClassifier(
  n_estimators=100,
  max_depth=5,
  random_state=42
```

) --> Bu yerda n_estimators=100 va max_depth=5 — hyperparameterlar.

7. Explainable AI va Non-Explainable AI

Explainable AI (XAI) — modelning bashoratlarini inson tushunishi uchun izohlashga qaratilgan yondashuvlar.

Daftardagi fikr:

> Rivojlanayotgan ishni, olinayotgan natijani va chiqarilayotgan modelning aniqligini hamda prediction qanday bo‘layotganini tushuntirib turadi. Ya'ni Explainable bo‘ladi.

Soddaroq qilib aytganda, Explainable AI modelning nima uchun aynan shunday bashorat berganini tushunishga yordam beradi.

Masalan, yurak kasalligi bashoratida SHAP orqali yosh, BMI yoki boshqa featurelarning model natijasiga qanday hissa qo‘shganini ko‘rish mumkin.

Non-Explainable / Black-box model — ichki qaror mexanizmini inson uchun tushunish qiyin bo‘lgan model.

Asosiy fikr:

Endi bunga qarama-qarshi ma'nodagi bitta tushuncha bor, o‘sha sanab o‘tkan ishlarimizni qanday qilinayotganini bilmaymiz biz.

Biroq murakkab model mutlaqo izohlab bo‘lmaydi degani emas. SHAP, LIME va boshqa usullar yordamida black-box modellarning bashoratlarini ham qisman izohlash mumkin.

Muhim: SHAP modelning bashoratiga feature qanday hissa qo‘shganini ko‘rsatadi, lekin sabab-oqibat munosabatini avtomatik isbotlamaydi.

7-dars: Feature Analysis in Supervised Machine Learning

1. Feature Analysis nima?

Feature Analysis (FA) — Machine Learning modelida ishlatiladigan input featurelarni o‘rganish, ularning target bilan bog‘liqligini, bir-biri bilan munosabatini va model natijasiga ta’sirini tekshirish jarayoni.

Datasetda juda ko‘p featurelar bo‘lishi mumkin. Lekin ularning hammasi foydali bo‘lavermaydi.

Foydasiz yoki muammoli featurelar

Feature turi	Muammo	Misol
Irrelevant	Modelga foydasiz ma’lumot qo‘shadi, noise yaratishi mumkin	Gol → ovqat
Highly correlated	Takroriy ma’lumot va multicollinearity keltirib chiqarishi mumkin	Jami gol va har bir o‘yindagi gol
Nonlinear / Complex	Oddiy model murakkab munosabatni ushlay olmasligi mumkin	Gol → futbolchining charchashi

Irrelevant features modelning accuracy ko‘rsatkichini pasaytirishi mumkin, lekin har doim emas.

Highly correlated features bir-biriga o‘xshash ma’lumot beradi. Bu ayniqsa linear model koeffitsiyentlarini izohlashda muammo tug‘dirishi mumkin.

Nonlinear relationships oddiy linear modellar tomonidan to‘liq o‘rganilmasligi mumkin.

2. Feature Analysisning asosiy maqsadlari

Feature Analysis quyidagi savollarga javob topishga yordam beradi:

1. Which features most affect the target?
2. Are some features redundant or correlated with others?
3. Should we transform any features?

Asosiy maqsadlar:

1. Target qiymatini bashorat qilishga yordam beradigan featurelarni aniqlash.
2. Irrelevant yoki redundant featurelarni topish.
3. Model performance, speed va interpretabilityni yaxshilash.

Muhim: Feature Analysis faqat target bilan eng kuchli correlationga ega featurelarni tanlash degani emas. Ayrim featurelar yakka holda kuchsiz bo‘lsa ham, boshqa featurelar bilan birgalikda foydali bo‘lishi mumkin.

3. Interpretability va Black-box Models

Interpretability — model natijasini inson tushuna olishi.

Masalan, tibbiy modelda qaysi featurelar predictionga ta’sir qilganini tushunish muhim.

Black-box model — ichki qaror qabul qilish jarayonini tushunish qiyin bo‘lgan model.

SHAP kabi metodlar murakkab modellarning predictionlarini izohlashga yordam beradi.

Misol:

Age → Spearman = 0.34

Bu age va target o‘rtasida musbat monotonic association mavjudligini bildiradi. Agar target heart disease mavjudligini bildirsa, kattaroq yosh guruhlarida kasallik ko‘proq uchrashi bilan mos kelishi mumkin. Ammo bu sabab-oqibatni isbotlamaydi.

4. Correlation with Target — Numeric Features

Numeric featurelarning target bilan bog‘liqligini tekshirish uchun:

Pearson correlation

Spearman correlation ishlatiladi.

4.1. Pearson Correlation

Pearson ikki o‘zgaruvchi orasidagi linear relationshipni o‘lchaydi.

$$r = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Belgi	Ma’nosi
r	Pearson correlation natijasi
X	Birinchi o‘zgaruvchi (masalan, Age)
Y	Ikkinchi o‘zgaruvchi (masalan, Blood Pressure)

Cov(X,Y)	Covariance — X va Y birgalikda qanday o‘zgarishini ko‘rsatadi
σX	X ning standard deviation qiymati
σY	Y ning standard deviation qiymati

Covariance nima?

Tasavvur qiling:

1. X oshganda Y ham oshishga moyil bo‘lsa, covariance musbat
2. X oshganda Y kamayishga moyil bo‘lsa, covariance manfiy.

Aniq birgalikdagi chiziqli o‘zgarish bo‘lmasa, covariance nolga yaqin bo‘lishi mumkin.

Standard deviation esa qiymatlar o‘z o‘rtachasidan qanchalik tarqalganligini ko‘rsatadi.

Pearson covariance qiymatini ikkita standard deviation ko‘paytmasiga bo‘ladi. Shu sababli natija −1 va +1 oralig‘ida bo‘ladi (ikkala o‘zgaruvchining standard deviationi noldan farqli bo‘lsa).

Faraz qilaylik, 3 nafar odamning yoshi va qon bosimi quyidagicha:

Odam	X: Age	Y: Blood Pressure
A	20	110
B	30	120
C	40	130

Bu misolda yosh 10 yilga oshganda qon bosimi ham 10 birlikka oshmoqda.

O‘rtacha qiymatlar:

$$\bar{X} = 30, \qquad \bar{Y} = 120$$

Sample covariance:

$$\mathrm{Cov}(X,Y) = \frac{(-10)(-10) + 0 + (10)(10)}{3-1} = 100$$

Sample standard deviation:

$$\sigma_X = 10, \qquad \sigma_Y = 10$$

Endi Pearson formulasiga qo‘yamiz:

$$r = \frac{100}{10 \times 10} = \boxed{1}$$

Natija: r = +1.0

Bu uchta nuqta mukammal musbat chiziqli bog‘liqlikka ega. Lekin bu yosh qon bosimi oshishining sababi ekanini isbotlamaydi.

codi
ng
qis
mi

```
import pandas as pd

# Select numeric features
numeric_features = df.select_dtypes(
    include=["int64", "float64"]
).drop(columns=["x6"], errors="ignore")

# Target
```

```
target = df["x6"]

# Pearson correlation
pearson_corr = numeric_features.corrwith(target)

print("Pearson Correlation with x6:\n")
print(pearson_corr)
```

4.2. Spearman Correlation

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

Spearman Correlation ikkita o‘zgaruvchi (X va Y) orasidagi monotonic relationshipni o‘lchaydi.

Pearsondan asosiy farqi: Spearman haqiqiy qiymatlar bilan emas, ularning tartibi (rank) bilan ishlaydi.

Masalan:

X = odamning yoshi (Age)

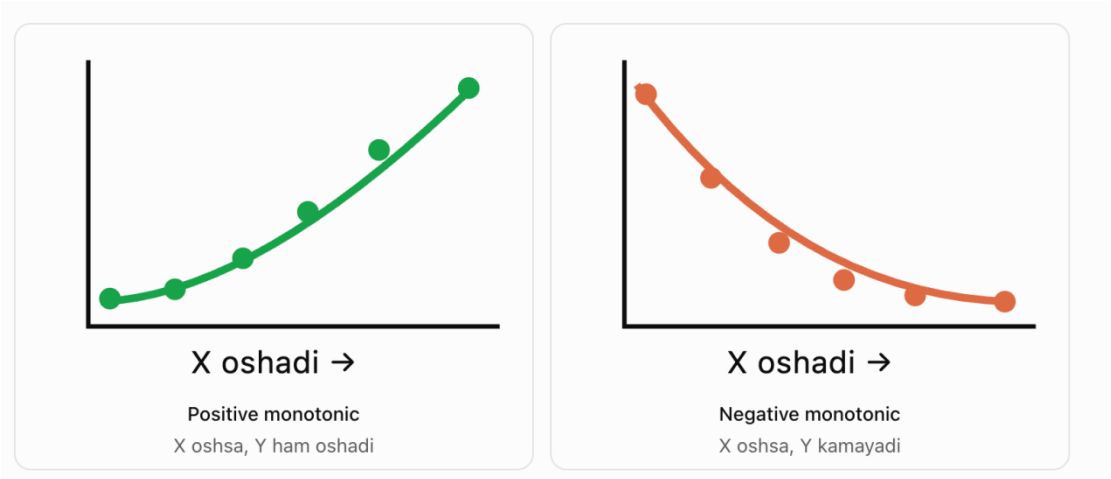
Y = qon bosimi (Blood Pressure)

Savol: Yoshi kattaroq odamlarda qon bosimi ham yuqoriroq bo‘lishga moyilmi?

Spearman shu savolga javob berishga yordam beradi.

1. Monotonic relationship nima?

Monotonic relationship — X oshgani sari Y bir yo‘nalishda o‘zgarishga moyil bo‘lishi.



Bu yerda chiziq to‘g‘ri bo‘lishi shart emas. Egri bo‘lsa ham, o‘sish yoki kamayish yo‘nalishi saqlansa, monotonic bo‘lishi mumkin.

Belgi	Manosi
p (rho)	Spearman correlation natijasi
d_i	X rank va Y rank orasidagi farq
d_i^2	Rank farqini kvadrati
E	Barcha qiymatlarni qo’shish
n	kuzatuvlar soni

Bu formula ranklar takrorlanmagan holat uchun ishlatiladi.

3. Haqiqiy sonlar bilan hisoblaymiz

Tasavvur qiling, 4 nafar odamning yoshi va qon bosimi quyidagicha:

Odam	Age (X)	Blood Pressure (Y)
A	20	110
B	30	130
C	40	120
D	50	150

1-qadam. Rank beramiz

Eng kichik qiymatga rank 1, keyingisiga rank 2 beriladi.

Odam	Age rank	BP rank
A	1	1
B	2	3
C	3	2
D	4	4

Ko‘ryapsizmi? B va C ning qon bosimi tartibi almashgan.
2-qadam. Rank farqini hisoblaymiz

$$d_i = Rank(X) - Rank(Y)$$

Odam	X rank	Y rank	d	d^2
A	1	1	0	0
B	2	3	−1	1
C	3	2	1	1
D	4	4	0	0
Jami				2

Demak:

$$\sum d_i^2 = 2, \qquad n = 4$$

Natija: Spearman = +0.8

Bu to‘rtta odam misolida kuchli musbat monotonic bog‘liqlik borligini bildiradi. Yosh kattalashganda qon bosimi ham odatda yuqoriroq, lekin tartib mukammal emas.

Codingda:

#

3-qadam. Formulaga qo‘yamiz

$$\rho = 1 - \frac{6 \times 2}{4(4^2 - 1)}$$

$$= 1 - \frac{12}{60}$$

$$\boxed{\rho = 0.8}$$

Spearman correlation

```
spearman_corr = numeric_features.corrwith(
   target,
   method="spearman"
)

print("\nSpearman Correlation with x6:\n")
print(spearman_corr)
```

.corr() va .corrwith() farqi:

.corr() — bir nechta numeric columnlar orasidagi correlation matrixni hisoblaydi.

.corrwith() — berilgan Series bilan har bir numeric column correlationini hisoblaydi.

5. Correlation with Target — Categorical Features

Categorical featurelarning target bilan bog‘liqligini tekshirish uchun konspektda quyidagi metodlar berilgan:

ANOVA

Chi-square test

1. ANOVA (Analysis of Variance) — bir nechta guruhning o‘rtacha qiymatlari bir-biridan statistik jihatdan farq qiladimi, shuni tekshiradi.

$$F = \frac{MS_{\text{between}}}{MS_{\text{within}}}$$

Belgi	Ma’nosi
F	ANOVA test statistic
MS_between	Guruhlar orasidagi o‘zgaruvchanlik
MS_within	Har bir guruh ichidagi o‘zgaruvchanlik

To‘liqroq:

$$MS_{\text{between}} = \frac{SS_{\text{between}}}{k-1}$$

$$MS_{\text{within}} = \frac{SS_{\text{within}}}{N-k}$$

Bu yerda k — guruhlar soni, N — jami kuzatuvlar soni, SS esa squared deviations yig‘indisi.

Uchta guruh talabalarning imtihon ballarini olaylik:

Guruh	Ballar	O‘rtacha
A	60, 65, 70	65
B	75, 80, 85	80
C	90, 95, 100	95

Savol: Uchta guruhning o‘rtacha ballari haqiqatan ham farq qiladimi?

ANOVA guruhlar orasidagi farqni guruhlarning ichidagi tarqalish bilan solishtiradi.

F katta bo‘lsa, guruhlar orasidagi farq ichki o‘zgaruvchanlikka nisbatan katta bo‘ladi. Statistik xulosa esa p-value va ANOVA shartlariga bog‘liq.

ANOVA coding:

```
import pandas as pd
from scipy.stats import f_oneway

# Select categorical features
categorical_features = df.select_dtypes(
    include=["object", "category"]
).columns

# Store ANOVA results
anova_results = {}

for col in categorical_features:

    groups = [
        df.loc[df[col] == val, "x6"].dropna()
        for val in df[col].dropna().unique()
    ]

    if len(groups) > 1:
        f_stat, p_val = f_oneway(*groups)

        anova_results[col] = {
            "F-statistic": f_stat,
            "p-value": p_val
        }

anova_df = pd.DataFrame(anova_results).T

anova_df = anova_df.sort_values("p-value")

print("ANOVA Results for Categorical Features vs x6:\n")
print(anova_df)
```

2.Chi-square($X^{2)}$ — ikkita categorical variable o‘rtasida statistik association bor yoki yo‘qligini tekshiradi.

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Belgi	Ma‘nosi
χ^2	Chi-square test statistic
O	Observed frequency — haqiqiy kuzatilgan son
E	Expected frequency — bog‘liqlik bo‘lmasa kutiladigan son
Σ	Barcha kataklar bo‘yicha yig‘indi

Expected frequency formulasi:

$$E = \frac{\text{Row Total} \times \text{Column Total}}{\text{Grand Total}}$$

Masalan:

100 kishining smoking va heart disease holatini olaylik:

Smoking	Heart Disease: Yes	Heart Disease: No	Total
Yes	30	20	50
No	10	40	50
Total	40	60	100

Masalan, Smoking = Yes va Heart Disease = Yes katagida:

Observed:

$$O = 30$$

Expected:

$$E = \frac{50 \times 40}{100} = 20$$

Ushbu katakning Chi-square hissasi:

$$\frac{(30 - 20)^2}{20} = 5$$

Qolgan uchta katakni ham hisoblaymiz:

$$\chi^2 = 5 + 3.33 + 5 + 3.33$$

$$\boxed{\chi^2 \approx 16.67}$$

Bu natija smoking va heart disease o‘rtasida association borligiga dalil bo‘lishi mumkin. Statistik ahamiyatni aniqlash uchun degrees of freedom va p-value ham kerak.

chi -square coding:

```
from scipy.stats import chi2_contingency

chi2_results = {}

for col in categorical_features:
   table = pd.crosstab(df[col], df["x6"])

   if table.shape[0] > 1 and table.shape[1] > 1:
      chi2, p_value, dof, expected = chi2_contingency(table)

      chi2_results[col] = {
```

```
        "Chi-square": chi2,
        "p-value": p_value
    }

chi2_df = pd.DataFrame(chi2_results).T
chi2_df = chi2_df.sort_values("p-value")

print(chi2_df)
```

Chi-square testda kichik p-value ikki categorical variable mustaqil emasligiga statistik dalil bo‘lishi mumkin. Ammo bu association kuchini bevosita o‘lchamaydi.

6. Correlation with Other Features

Featurelar orasidagi correlationni tekshirishning asosiy maqsadi:

1. Highly correlated featurelarni aniqlash
2. Redundant informationni topish.
3. Multicollinearity xavfini baholash.
4. Modelning generalization va interpretability jihatlarini yaxshilash

6.1. Seaborn Heatmap

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))

sns.heatmap(
   df[numeric_features.columns].corr(),
   annot=True,
   cmap="coolwarm"
)

plt.title("Feature-to-Feature Correlation")
plt.show()
```

6.2. Highly Correlated Featuresni aniqlash

Misol uchun correlation threshold: $|r| > 0.9$

```
import numpy as np

# Compute absolute correlation matrix
corr_matrix = df[numeric_features.columns].corr().abs()

# Select upper triangle
```

```
upper = corr_matrix.where(
    np.triu(
        np.ones(corr_matrix.shape),
        k=1
    ).astype(bool)
)

# Identify highly correlated features
to_drop = [
    column
    for column in upper.columns
    if any(upper[column] > 0.9)
]

print("Highly correlated features to consider dropping:")
print(to_drop)
```

Bu kod 0.9 dan yuqori correlationga ega featurelardan ayrimlarini chiqarib tashlash uchun nomzod sifatida ko‘rsatadi.

Lekin avtomatik ravishda drop qilish har doim to‘g‘ri emas.

Masalan, ikkita feature juda correlated bo‘lsa:

1. Qaysi biri targetni yaxshiroq bashorat qilishini tekshiramiz.
2. Domain knowledge asosida muhimrog‘ini baholaymiz.
3. Feature engineering yordamida yangi feature yaratish imkoniyatini ko‘ramiz.
4. Model performance orqali feature removal foydali yoki yo‘qligini tekshiramiz.

Buni bajarishda feature selection faqat training data ichida amalga oshirilishi kerak.

7. Interactive Heatmap with Plotly Express

Plotly yordamida interaktiv correlation heatmap yaratish mumkin.

```
import plotly.express as px
import numpy as np
import pandas as pd

# Compute correlation matrix
corr_matrix = df[numeric_features.columns].corr()

# Interactive heatmap
fig = px.imshow(
    corr_matrix,
    text_auto=True,
    color_continuous_scale="RdBu_r",
```

```
    origin="upper",
    labels=dict(
        x="Features",
        y="Features",
        color="Correlation"
    ),
    title="Feature-to-Feature Correlation (Numeric Features)"
)

fig.update_xaxes(side="top")

fig.show()
```

FEATURE IMPORTANCE FROM MODELS

8. Model-based Feature Importance

Correlation Analysis va Model-based Feature Importance bir xil tushuncha emas.

Correlation Analysis — featurelar orasidagi statistik bog‘liqlikni o‘rganadi.

Model-based Feature Importance — model o‘rgangan prediction jarayonida featurelardan qanday foydalanilganini baholaydi.

Asosiy yondashuvlar:

Model	Feature Importance usuli
Linear Regression	Coefficients
Logistic Regression	Coefficients
Random Forest	Gini / Impurity-based importance (classification)
Tree-based Regression	Impurity reduction
Complex Models	SHAP, Permutation Importance

9. Linear Regression Coefficients

Linear modelda coefficient feature qiymati o‘zgarganda prediction qanday o‘zgarishini ko‘rsatadi, boshqa featurelar o‘zgarmas deb qaralganda.

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

X = numeric_features
y = target

# Scale numeric features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Train model
lr = LinearRegression()
lr.fit(X_scaled, y)

# Feature coefficients
coefficients = pd.Series(
    lr.coef_,
    index=X.columns
).sort_values(
    key=abs,
    ascending=False
)

print("Linear Regression Feature Coefficients:\n")
print(coefficients)
```

lr.coef_ har bir feature coefficientini qaytaradi.

sort_values(key=abs, ascending=False) coefficientlarning absolute qiymati bo‘yicha tartiblaydi.

10. Random Forest Feature Importance

Random Forest feature importance qiymatlarini modeldagi daraxtlarning splitlaridan hisoblaydi.

```
from sklearn.ensemble import RandomForestRegressor
import pandas as pd

rf = RandomForestRegressor(random_state=42)

rf.fit(X, y)

# Feature importance
feat_importance = pd.Series(
    rf.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print("Random Forest Feature Importance:\n")
print(feat_importance)
```

Impurity-based importance correlated featurelar va ko‘p noyob qiymatli featurelar sababli chalg‘ituvchi bo‘lishi mumkin.

Feature Analysis natijasini ikki xil solishtirish

1. Feature Analysis qilmasdan model training.
2. Feature Analysis va Feature Engineeringdan keyin model training.

PERFORMANCE IMPACT

11. Feature Removal Model Performancega qanday ta’sir qiladi?

Asosiy maqsad — bitta feature olib tashlanganda model performance qanday o‘zgarishini ko‘rish.

Misol:

1. Full Model — barcha featurelar bilan.
2. Reduced Model — shots feature olib tashlangan.

Regression metrics:

1. R^2 — yuqori bo‘lsa yaxshi.
2. RMSE — past bo‘lsa yaxshi.

```
from sklearn.model_selection import train_test_split

from sklearn.metrics import r2_score, mean_squared_error

from sklearn.ensemble import RandomForestRegressor

import numpy as np

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=42)

rf_full = RandomForestRegressor(random_state=42)

rf_full.fit(X_train, y_train)

y_pred_full = rf_full.predict(X_test)

full_rmse = np.sqrt( mean_squared_error( y_test, y_pred_full ))

full_r2 = r2_score(y_test, y_pred_full)

X_train_reduced = X_train.drop(columns=["shots"])

X_test_reduced = X_test.drop(columns=["shots"])
```

```
rf_reduced = RandomForestRegressor(random_state=42)

rf_reduced.fit(X_train_reduced, y_train)

y_pred_reduced = rf_reduced.predict(X_test_reduced)

reduced_rmse = np.sqrt( mean_squared_error(y_test, y_pred_reduced))

reduced_r2 = r2_score(y_test, y_pred_reduced)

print(f"Full model R²: {full_r2:.3f}, " f"RMSE: {full_rmse:.3f}")

print(f"Reduced model R²: {reduced_r2:.3f}, " f"RMSE: {reduced_rmse:.3f}")
```

Masalan, quyidagi qiymatlar faqat tushuntirish uchun

Model	R^2	RMSE
Full Model	0.85	4.20
Reduced Model	0.87	3.90

Reduced Model yaxshiroq natija bergan. Demak, shots feature olib tashlanganda shu baholashda performance yaxshilangan.

Lekin bir martalik test natijasi bilan umumiy xulosa chiqarib bo‘lmaydi. Ishonchliroq qaror uchun cross-validationdan foydalaniladi.

12. Feature Engineering bilan bog‘liqligi

Feature Analysis — mavjud featurelarni o‘rganish.

Feature Engineering (FE) — mavjud ma’lumotlardan yangi yoki o‘zgartirilgan featurelar yaratish.

Masalan:

Total Goals va Matches Played featurelaridan:

$$\text{Goals per Match} = \frac{\text{Total Goals}}{\text{Matches Played}}$$

yangi feature yaratish mumkin.

Agar ikkita feature highly correlated bo‘lsa, ulardan bittasini darhol olib tashlash o‘rniga, ularning ratio yoki boshqa transformatsiyasi foydali bo‘lishi mumkinligini tekshirish kerak.

8-dars:SHAP Value

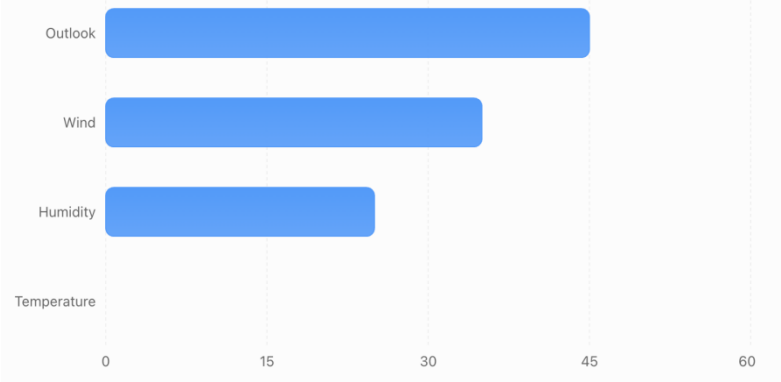
1. Feature Importance nima?

Feature Importance (FI) — Machine Learning modelida har bir feature (xususiyat) prediction qilishda qanchalik muhim ekanligini baholaydi.

Masalan, ob-havo modelida quyidagi featurelar bo‘lishi mumkin:

1. Outlook — ob-havo holati
2. Wind — shamol
3. Humidity — namlik
4. Temperature — harorat

Feature Importance — tushuntirish uchun namunaviy qiymatlar



Bu misolda Outlook eng muhim feature, Temperature esa tanlangan importance usuli bo‘yicha ahamiyatsiz yoki deyarli ishlatilmagan.

Feature Importance usullari

1. Tree-based models

model.feature_importances_

Decision Tree, Random Forest va ayrim boosting modellarida ishlatiladi.

3. Linear models

model.coef_

Linear Regression yoki Logistic Regression kabi modellarda koeffitsiyentlarni olish uchun ishlatiladi.

Koeffitsiyentning musbat yoki manfiy bo‘lishi prediction yo‘nalishini bildiradi. Ammo koeffitsiyent kattaliklarini taqqoslashda featurelarning o‘lchov birligi va scaling holatini hisobga olish kerak.

2. Feature Importance limitations

Feature Importance quyidagi cheklovlarga ega:

1. Ko‘plab importance usullari feature predictionni qaysi tomonga o‘zgartirganini ko‘rsatmaydi.
2. Odatda alohida sample uchun prediction qanday hosil bo‘lganini tushuntirmaydi.
3. Highly correlated featurelar mavjud bo‘lsa, importance noto‘g‘ri talqin qilinishi mumkin.
4. Importance yuqori bo‘lishi feature predictionga sababiy ta’sir ko‘rsatishini anglatmaydi.

3. SHAP nima?

SHAP — SHapley Additive exPlanations.

SHAP Machine Learning modelining predictionini tushuntirish uchun ishlatiladi.

U har bir feature modelning baseline predictionidan qanchalik va qaysi yo‘nalishda chetlanishiga hissa qo‘shganini ko‘rsatadi.

Asosiy formula

$$f(x) = E[f(X)] + \sum_{i=1}^{M} \phi_i$$

Bu yerda:

- $f(x)$ — yakuniy model predictioni.
- $E[f(X)]$ — baseline (expected prediction).
- ϕ_i — i-feature uchun SHAP value.
- M — featurelar soni.

Demak:

$$\text{Final Prediction} = \text{Baseline} + \sum \text{SHAP Values}$$

4. SHAP qanday ishlaydi?

Masalan, model futbolchining reytingini bashorat qilmoqda.

Namunaviy SHAP hisoblash

Baseline rating	**70**
Goals	+10
Passes	−5
Assists	+3
Final predicted rating	**78**

$$70 + 10 - 5 + 3 = 78$$

1. Positive SHAP → predictionni oshiradi.
2. Negative SHAP → predictionni kamaytiradi.
3. Zero SHAP → baselinega nisbatan hissa nol.

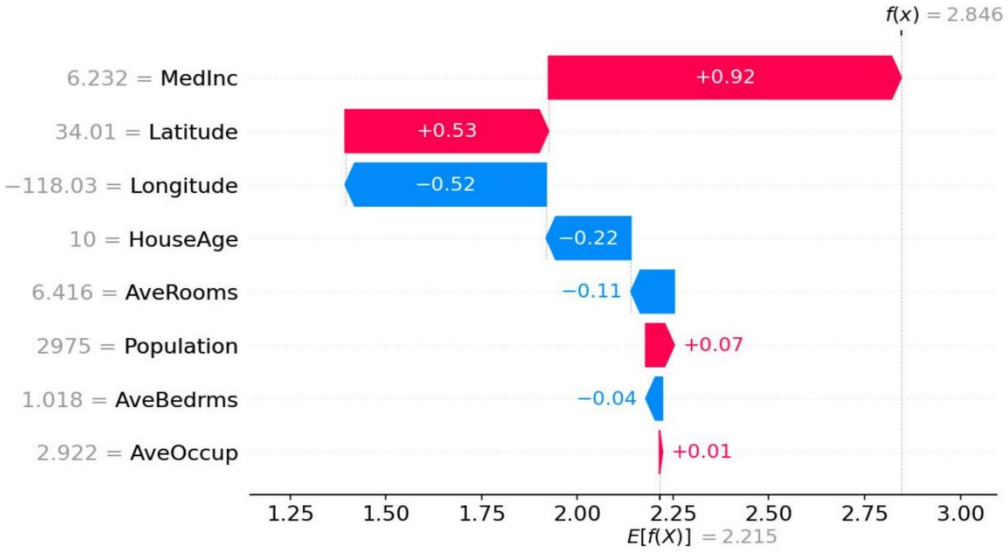
Muhim: Positive SHAP har doim yaxshi, negative SHAP esa har doim yomon degani emas. Ular faqat model predictionining yo‘nalishini bildiradi.

5. Global va Local SHAP

Global SHAP	Local SHAP
Barcha samplelar bo‘yicha feature importance	Bitta sample predictioni
Qaysi featurelar umumiy muhimligini ko‘rsatadi	Qaysi feature predictionni oshirgan yoki kamaytirganini ko‘rsatadi
Summary / beeswarm plot	Waterfall plot

Global importance ko‘pincha mean absolute SHAP bilan hisoblanadi:

$$I_j = \frac{1}{n}\sum_{i=1}^{n} |\phi_{ij}|$$



Waterfall plot bitta sample uchun baseline qiymatdan yakuniy predictiongacha qanday yetib borilganini ko‘rsatadi.

Masalan, yurak kasalligi prediction modelida:

1. Baseline probability = 8%
2. Age contribution = +6 percentage points
3. Poor General Health = +9 percentage points

4. Boshqa featurelar = −3 percentage points

Final prediction = 20%.

Bu faqat tushuntirish uchun misol. Haqiqiy SHAP qiymatlarini qo‘shishda model outputining birligi bir xil bo‘lishi kerak. Masalan, log-odds uchun hisoblangan SHAP qiymatlarini bevosita probability foizlariga qo‘shib bo‘lmaydi.

SHAP limitations

SHAP juda foydali, ammo:

1. Causality (sabab-oqibat) ni isbotlamaydi.
2. Correlated featurelar natijalarni talqin qilishni qiyinlashtirishi mumkin.
3. Background data tanlovi natijalarga ta’sir qilishi mumkin.
4. Katta dataset va murakkab modelda hisoblash ko‘p vaqt oladi.

```
import pandas as pd

import numpy as np

import shap

import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.model_selection import train_test_split

from sklearn.pipeline import Pipeline

from sklearn.compose import ColumnTransformer

from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import r2_score, mean_squared_error


df_copy = df.copy()

# Features and target

X = df_copy.drop(columns=["X6"], errors="ignore")

y = df_copy["X6"]

# Numeric features

numeric_features = X.select_dtypes(
```

```
    include=["int64", "float64"]
).columns.tolist()
# Categorical features
categorical_features = X.select_dtypes(
    include=["object", "category"]
).columns.tolist()
# label encoding
def label_encode_df(df, cat_cols):
    df_encoded = df.copy()
    encoders = {}
    for col in cat_cols:
        le = LabelEncoder()
        df_encoded[col] = le.fit_transform(
            df_encoded[col].astype(str)
        )
        encoders[col] = le
    return df_encoded, encoders


    X_encoded, encoders = label_encode_df(X,categorical_features)
```

Bu funksiya categorical qiymatlarni raqamlarga aylantiradi.

Original	Encoded
Sunny	2
Rainy	1
Cloudy	0

Amaliy ML workflowda nominal featurelar uchun ko‘pincha **OneHotEncoder** yaxshiroq. Bundan tashqari, encoder train/test splitdan oldin fit qilinsa, leakage xavfi paydo bo‘ladi. Shu sababli haqiqiy loyiha uchun encodingni pipeline ichida train ma’lumotida fit qilish kerak.

```
X_train, X_test, y_train, y_test =
train_test_split(X_encoded,y,test_size=0.2,random_state=42)

preprocessor = ColumnTransformer(

   transformers=[

      (

         "num",

         StandardScaler(),

         numeric_features

      )

   ],

   remainder="passthrough"

)



overfit_model = Pipeline([

   ("preprocessor", preprocessor),

   (

      "regressor",

      DecisionTreeRegressor(

         max_depth=None,

         random_state=42

      )

   )

])
```

max_depth=None daraxt chuqurligini oldindan cheklamaydi. Bunday model training data ustida overfitting qilishi mumkin.

```
overfit_model.fit(X_train, y_train)
```

```
y_train_pred = overfit_model.predict(X_train)

y_test_pred = overfit_model.predict(X_test)

print( "Train R²:", r2_score(y_train, y_train_pred))

print("Test R²:",r2_score(y_test, y_test_pred))

print("Test MSE:", mean_squared_error(y_test, y_test_pred))
```

Agar Train R^2 juda yuqori, Test R^2 esa ancha past bo‘lsa, bu overfitting belgisi bo‘lishi mumkin.

SHAP TreeExplainer

```
X_test_preprocessed = (overfit_model.named_steps["preprocessor"].transform(X_test))

tree_model = overfit_model.named_steps["regressor"]

explainer = shap.TreeExplainer(tree_model)

shap_values = explainer.shap_values(X_test_preprocessed)
```

Waterfall plot

```
feature_names = (overfit_model.named_steps["preprocessor"].get_feature_names_out())

sample_id = 0

base_value = float(np.asarray(explainer.expected_value).item())


explanation = shap.Explanation(
   values=shap_values[sample_id],
   base_values=base_value,
   data=X_test_preprocessed[sample_id],
   feature_names=feature_names
)

shap.plots.waterfall(explanation)
```

Summary plot

```
shap.summary_plot(shap_values,X_test_preprocessed,feature_names=feature_names)
```

Linear Regression coefficients

Linear Regression modeli:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

Bu yerda:

- β_0 — intercept.
- β_i — feature coefficient.
- x_i — input feature.
- $\hat{y}$ — predicted value.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X, y)

print("Model coefficients:")

for i in range(X.shape[1]):

    print(X.columns[i],"=",round(model.coef_[i], 5))
```

Linear Model SHAP

```
    import shap

    X, y = shap.datasets.california(n_points=1000)


X100 = shap.utils.sample(X,100,random_state=42)


model = LinearRegression()

model.fit(X, y)

explainer = shap.Explainer(model.predict,X100)

shap_values = explainer(X)
```

Partial Dependence Plot

Partial Dependence Plot (PDP) model predictioni tanlangan feature qiymati o‘zgarganda o‘rtacha qanday o‘zgarishini ko‘rsatadi.

```
from sklearn.inspection import PartialDependenceDisplay
```

```
PartialDependenceDisplay.from_estimator(model,X100,features=["MedInc"],kind="average")

plt.show()
```

MedInc — median income.

PDP feature qiymati bilan model predictioni o‘rtasidagi o‘rtacha bog‘lanishni ko‘rsatadi, ammo bu causal effect emas.

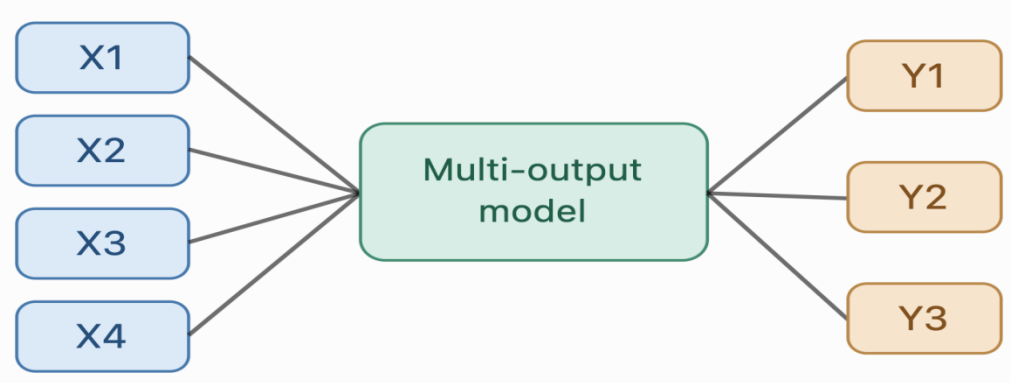
Bitta sample uchun SHAP dependence ko‘rinishi:

```
sample_id = 20

shap.plots.scatter(shap_values[:, "MedInc"])

shap.plots.waterfall(shap_values[sample_id],max_display=14)
```

19. Multi-Output nima?
Multi-output model — bitta input orqali bir nechta targetni bashorat qiladigan model.



Multi-Class vs Multi-Label

Multi-Class	Multi-Class
Multi-Label	Multi-Label
Har bir sample bitta classga tegishli	Har bir sample bitta classga tegishli
Bitta sample bir nechta labelga tegishli bo‘lishi mumkin	Bitta sample bir nechta labelga tegishli bo‘lishi mumkin

Multi-Output Regression

Multi-output regressionda har bir target numeric qiymat bo‘ladi.

Masalan, futbolchi uchun:

1. Goals — nechta gol uradi?
2. Assists — nechta assist qiladi?
3. xG — expected goals.

```
from sklearn.multioutput import MultiOutputRegressor
```

```
from sklearn.ensemble import RandomForestRegressor

base_model = RandomForestRegressor(random_state=42)

multi_output_model = MultiOutputRegressor(base_model)
```

MultiOutputRegressor har bir target uchun alohida regressor fit qiladi. U targetlar orasidagi bog‘lanishni bevosita modellashtirmaydi.

Multi-Output Classification

```
from sklearn.multioutput import MultiOutputClassifier

from sklearn.ensemble import RandomForestClassifier

base_model = RandomForestClassifier(random_state=42)

multi_output_classifier = MultiOutputClassifier(base_model)
```

Har bir target categorical bo‘ladi. Targetlarning class soni turlicha bo‘lishi mumkin.

MULTI-OUTPUT REGRESSION

```
import pandas as pd

import numpy as np

import joblib


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import OneHotEncoder, MinMaxScaler

from sklearn.compose import ColumnTransformer

from sklearn.pipeline import Pipeline

from sklearn.multioutput import MultiOutputRegressor

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_squared_error, r2_score


target_cols = ["goals", "xG"]

X = df.drop(
```

```
    columns=target_cols + ["id", "player_name"],
    errors="ignore"
)
y = df[target_cols]
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)
# numeric and categorical features
categorical_features = [
    "position",
    "team_title"
]
numerical_features = [
    col for col in X.columns
    if col not in categorical_features
]
# Numerica transofrmation
numeric_transformer = Pipeline([
    ("scaler", MinMaxScaler())
])
# Categorical transformer
categorical_transformer = Pipeline([
    (
        "encoder",
        OneHotEncoder(
            handle_unknown="ignore"
        )
```

```
    )
])
# ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            numeric_transformer,
            numerical_features
        ),
        (
            "cat",
            categorical_transformer,
            categorical_features
        )
    ]
)
# Base model
base_model = RandomForestRegressor(
    random_state=42
)
multi_output_model = MultiOutputRegressor(
    base_model
)
# Full pipeline
```

```
pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", multi_output_model)
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
for i, target in enumerate(target_cols):
    mse = mean_squared_error(
        y_test[target],
        y_pred[:, i]
    )
    r2 = r2_score(
        y_test[target],
        y_pred[:, i]
    )
    print(
        f"{target} => MSE: {mse:.4f}, R²: {r2:.4f}"
    )
    # save model
    model_filename = "multioutput_goals_xG_model.joblib"
    joblib.dump(
        pipeline,
        model_filename
    )
    print(
```

```
    f"Model saved successfully as {model_filename}"
)
```

Offline test

```
user_input = [
  {
    "id": 1,
    "player_name": "Erling Haaland",
    "games": 30,
    "time": 2500,
    "assists": 5,
    "shots": 90,
    "key_passes": 25,
    "npxG": 25.7,
    "xA": 4.7,
    "xGChain": 28.9,
    "xGBuildup": 6.3,
    "league": "EPL",
    "position": "ST",
    "yellow_cards": 2,
    "red_cards": 0,
    "team_title": "Manchester City",
    "npg": 2,
    "actual_goals": 27,
    "actual_xG": 26.8,
  },
  {
    "id": 2,
    "player_name": "Kylian Mbappé",
```

```
    "games": 28,
    "time": 2400,
    "assists": 6,
    "shots": 88,
    "key_passes": 28,
    "npxG": 23.8,
    "xA": 5.8,
    "xGChain": 27.4,
    "xGBuildup": 5.9,
    "league": "Ligue1",
    "position": "ST",
    "yellow_cards": 1,
    "red_cards": 0,
    "team_title": "PSG",
    "npg": 1,
    "actual_goals": 25,
    "actual_xG": 24.5,
},
{
    "id": 3,
    "player_name": "Harry Kane",
    "games": 32,
    "time": 2800,
    "assists": 4,
    "shots": 84,
    "key_passes": 20,
    "npxG": 21.0,
    "xA": 4.3,
    "xGChain": 23.5,
    "xGBuildup": 4.7,
```

```
    "league": "Premier League",
    "position": "ST",
    "yellow_cards": 3,
    "red_cards": 0,
    "team_title": "Tottenham",
    "npg": 2,
    "actual_goals": 22,
    "actual_xG": 21.9,
  }
]

df_input = pd.DataFrame(user_input)
```

Load the model and predict

```
loaded_pipeline = joblib.load(
  "multioutput_goals_xG_model.joblib"
)

required_cols = X_train.columns.tolist()
```

```
for col in required_cols:
  if col not in df_input.columns:
```

```
    if col in categorical_features:
      df_input[col] = "Unknown"
    else:
      df_input[col] = 0
```

```
X_features = df_input[required_cols]
```

```
predictions = loaded_pipeline.predict(
  X_features
```

```
)
```

Show the result

```
df_results = df_input[
    ["id", "player_name"]
].copy()


df_results["Predicted Goals"] = predictions[:, 0]
df_results["Predicted xG"] = predictions[:, 1]
```

```
print("\nPredictions for 3 players:")
print(df_results)
```

RMSE Calculation

Agar yangi futbolchilar uchun haqiqiy goals va xG qiymatlari mavjud bo‘lsa:

```
rmse_goals = np.sqrt(
    mean_squared_error(
        df_input["actual_goals"],
        df_results["Predicted Goals"]
    )
)


rmse_xG = np.sqrt(
    mean_squared_error(
        df_input["actual_xG"],
        df_results["Predicted xG"]
    )
)
```

```
print(f"RMSE for Goals: {rmse_goals:.2f}")
print(f"RMSE for xG: {rmse_xG:.2f}")
```

actual_goals va actual_xG ustunlari bo‘lmasa, bu oxirgi kodni ishlatib bo‘lmaydi.

Concept	Main purpose
Feature Importance	Qaysi featurelar model uchun muhim?
SHAP Global	Qaysi featurelar umumiy predictionlarda ko‘proq hissa qo‘shadi?
SHAP Local	Nega model aynan shu sample uchun bunday prediction qildi?
Multi-Class	Bir nechta classdan bittasini tanlash
Multi-Label	Bir samplega bir nechta label berish
Multi-Output Regression	Bir vaqtda bir nechta numeric targetni bashorat qilish
Multi-Output Classification	Bir vaqtda bir nechta categorical targetni bashorat qilish

In summaryda:

Feature Importance: Model umumiy predictionlarda qaysi featurelardan ko‘proq foydalanadi yoki qaysilariga ko‘proq tayanadi?

SHAP: Har bir feature predictionni baselinega nisbatan qaysi tomonga va qanchaga suradi?

Positive SHAP: Prediction qiymatini oshiradi.

Negative SHAP: Prediction qiymatini kamaytiradi.

Waterfall Plot: Bitta predictionning baseline qiymatdan final qiymatgacha qanday shakllanganini ko‘rsatadi.

Multi-Output Regression: Bitta inputdan bir nechta numeric prediction olish.

Multi-Output Classifier: Bitta inputdan bir nechta categorical prediction olish.

Pipeline: Preprocessing va modelni bitta workflow ichida birlashtirish.

Overfitting: Model training data uchun juda yaxshi, yangi data uchun esa yomon ishlashi.

RMSE: Prediction xatolarining o‘rtacha kvadratik ildizi; kichikroq qiymat odatda yaxshiroq.

9-dars: Optimization

1. Optimization nima?

Optimization - Machine Learning modelining ishlashini yaxshilash uchun eng mos parametrlar va sozlamalarni topish jarayoni.

Murakkab datasetlarda bitta algoritm har doim ham eng yaxshi natija bermaydi. Chunki ma’lumotlarda nonlinear relationships, noise, outliers va murakkab patterns bo‘lishi mumkin.

Optimization maqsadi:

1. Errorni kamaytirish.
2. Accuracy va boshqa muhim metrikalarni yaxshilash.
3. Underfitting va overfitting xavfini kamaytirish.
4. Generalization va stabilityni yaxshilash.
5. Hisoblash resurslaridan samarali foydalanish.

Optimization turlari

Model Parameters	Hyperparameters
Training vaqtida o‘rganiladi	Trainingdan oldin belgilanadi
Weights, coefficients, bias	max_depth, learning_rate, n_estimators
Optimizer orqali yangilanadi	Manual search, GridSearchCV, RandomizedSearchCV va boshqalar

Optimization har doim accuracy oshishini yoki training time qisqarishini kafolatlamaydi. Natija dataset, model va tanlangan sozlamalarga bog‘liq.

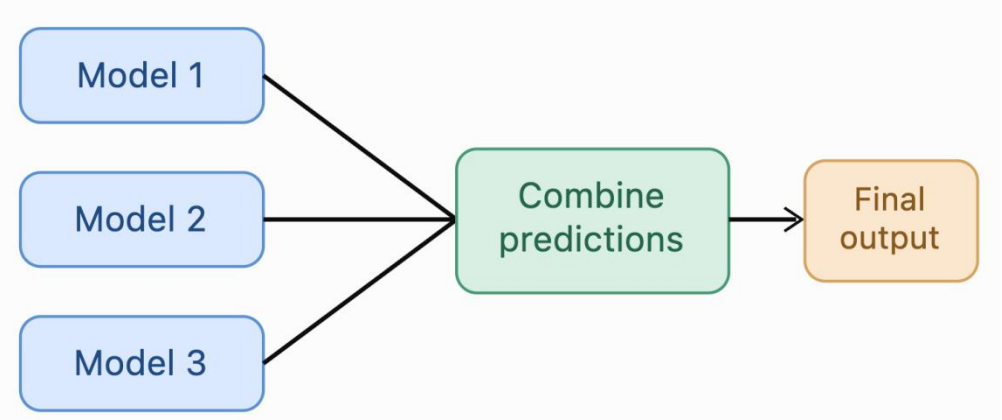
2. Ensemble Learning nima?

Ensemble Learning — bir nechta modelning predictionlarini birlashtirib, bitta yakuniy prediction olish usuli.

$$\text{Ensemble} = M_1 + M_2 + \cdots + M_n$$

Bu
yerda + oddiy qo‘shishni emas, predictionlarni birlashtirishni anglatadi.

Bu yerda + oddiy qo‘shishni emas, predictionlarni birlashtirishni anglatadi.



Asosiy afzalliklari

1. Accuracy: Prediction sifatini oshirishi mumkin.
2. Stability: Natijalar barqarorroq bo‘lishi mumkin.
3. Robustness: Noise, outliers va ayrim data o‘zgarishlariga chidamlilikni oshirishi mumkin.
4. Generalization: Yangi ma’lumotlarda yaxshiroq ishlashi mumkin.

Robustness — model noise, outliers yoki distribution shift mavjud bo‘lganda ham ishlash sifatini qanchalik saqlab qolishi.

Noise — ma’lumotdagi tasodifiy xatolik yoki keraksiz o‘zgarishlar. Outlier — boshqalardan keskin farq qiluvchi qiymat. Distribution shift — training va yangi ma’lumotlar taqsimotining farqlanishi.

3. Ensemble Learning turlari

Method	How it works	Examples
Bagging	Modellar bootstrap samplelarda ko‘pincha parallel o‘qitiladi	Random Forest, BaggingRegressor
Boosting	Modellar ketma-ket qurilib, oldingi xatolarni kamaytirishga intiladi	AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost
Voting	Bir nechta model predictionlari birlashtiriladi	VotingClassifier, VotingRegressor
Stacking	Base model predictionlarini meta-model birlashtiradi	StackingClassifier

VOTING

Voting nima?

Voting — turli base learnerlarning predictionlarini birlashtirish usuli.

Masalan:

1. Logistic Regression (LR)
2. Decision Tree (DT)
3. Support Vector Classifier (SVC)

Ular bir xil training datasetda o‘qitilib, yakuniy prediction birgalikda hosil qilinadi.

Classification uchun ikkita usul mavjud:

1. **Hard Voting:** Class label bo‘yicha ko‘pchilik ovozi.
2. **Soft Voting:** Class probabilitylarni o‘rtachalashtirish.

Hard Voting

Hard Voting — eng ko‘p ovoz olgan classni tanlaydi.

Algorithm	Prediction
Logistic Regression	Yes
Decision Tree	No
SVC	No
Final Prediction	No

mode — eng ko‘p uchragan qiymat.

Soft Voting

$$\hat{y} = \text{mode}(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_m)$$

Soft Voting — har bir modelning probability qiymatlarini o‘rtachalashtiradi.

Model	P(Yes)	P(No)
LR	0.70	0.30
DT	0.60	0.40
SVC	0.40	0.60

$$P(Yes) = \frac{0.70 + 0.60 + 0.40}{3} = 0.567$$

$$P(No) = \frac{0.30 + 0.40 + 0.60}{3} = 0.433$$

Final prediction = Yes, chunki 0.567 > 0.433.

Soft Voting uchun modellarda predict_proba() bo‘lishi kerak. SVC uchun probability=True belgilanadi.

Muhim: Dataset balanced bo‘lsa Soft Voting, imbalanced bo‘lsa Hard Voting albatta yaxshi degan qoida yo‘q. Imbalanced datasetlarda Recall, AP, F1, calibration va thresholdni tekshirish kerak.

VOTING CLASSIFIER : Coding

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.ensemble import VotingClassifier
```

Feature, Target and Train/Test Split

```
X = df.drop("red_cards", axis=1)
y = df["red_cards"]


X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
    test_size=0.3,
    random_state=42
)
```

Base learners

```
lr = LogisticRegression(max_iter=1000)
dt = DecisionTreeClassifier(random_state=42)
svm = SVC(probability=True)
```

Hard Voting

```
hard_voting = VotingClassifier(
    estimators=[
        ("lr", lr),
        ("dt", dt),
        ("svm", svm)
    ],
    voting="hard"
)

hard_voting.fit(X_train, y_train)
y_pred_hard = hard_voting.predict(X_test)
print(
    "Hard Voting Accuracy:",
    accuracy_score(y_test, y_pred_hard)
)
```

Soft Voting

```
soft_voting = VotingClassifier(
    estimators=[
```

```
        ("lr", lr),
        ("dt", dt),
        ("svm", svm)
    ],
    voting="soft"
)

soft_voting.fit(X_train, y_train)
y_pred_soft = soft_voting.predict(X_test)
print(
    "Soft Voting Accuracy:",
    accuracy_score(y_test, y_pred_soft)
)
print("\nSoft Voting Probabilities:")
print(soft_voting.predict_proba(X_test[:5]))
```

predict_proba() dastlabki beshta sample uchun har bir class ehtimolini ko‘rsatadi.

Voting Regressor

Regressionda class label emas, numeric predictionlar o‘rtachasi olinadi.

Masalan, uchta model uy narxini bashorat qildi:

1. Linear Regression = $200,000
2. Ridge = $220,000
3. Decision Tree = $210,000

$$\hat{y} = \frac{200000 + 220000 + 210000}{3} = 210000$$

```
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.ensemble import VotingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
```

Features and Target

```
X = df.drop("X6", axis=1)
y = df["X6"]


X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42
)
```

Base Models for Voting Regresso

```
lr = LinearRegression()
ridge = Ridge()
dt = DecisionTreeRegressor(random_state=42)
voting_reg = VotingRegressor(
    estimators=[
        ("lr", lr),
        ("ridge", ridge),
        ("dt", dt)
    ]
)


voting_reg.fit(X_train, y_train)
y_pred = voting_reg.predict(X_test)
```

Evaluation

```
print(
    "Voting Regressor MSE:",
    mean_squared_error(y_test, y_pred)
)


print(
    "Voting Regressor R²:",
    r2_score(y_test, y_pred)
)
print("\nFirst 5 Predictions:")
print(y_pred[:5])
```

MSE — Mean Squared Error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Kichikroq MSE odatda yaxshiroq.

R² — Coefficient of Determination

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

R² 1 ga yaqin bo'lsa, model ma'lumotdagi o'zgaruvchanlikni yaxshiroq tushuntiradi. R² manfiy ham bo'lishi mumkin.

Concept	Purpose
Optimization	Model sozlamalarini yaxshilash
Parameters	Training jarayonida o'rganiladigan qiymatlar
Hyperparameters	Trainingdan oldin tanlanadigan sozlamalar
Ensemble Learning	Bir nechta modelni birlashtirish
Bagging	Bootstrap samplelar orqali modellarni o'qitish
Boosting	Ketma-ket modellar bilan xatolarni kamaytirish
Hard Voting	Majority class
Soft Voting	Average probability
Voting Regressor	Average numeric prediction
Robustness	Shovqin va o'zgarishlarga chidamlilik
Generalization	Yangi data ustida yaxshi ishlash

10-dars:Ensemble Learning-Bagging, Boostig, Voting and Stacking

1. Ensemble Learning nima?

Ensemble Learning — bir nechta Machine Learning modellarini birlashtirib, yaxshiroq natija olish usuli.

1. Linear Learning: chiziqli munosabatlarni o‘rganadigan modellar, masalan, Linear Regression.
2. Ensemble Learning: bir nechta model natijalarini birlashtiradigan yondashuv.

Nima uchun kerak?

Bitta model (masalan, Decision Tree yoki Linear Regression) ba'zan overfitting qiladi yoki yetarlicha yaxshi natija bermaydi.

Ensemble usullari quyidagilarni yaxshilashi mumkin:

1. Accuracy — bashorat aniqligi.
2. Robustness — shovqin, outlier va ma'lumot o‘zgarishlariga chidamlilik.
3. Generalization — yangi ma'lumotlarda yaxshi ishlash.

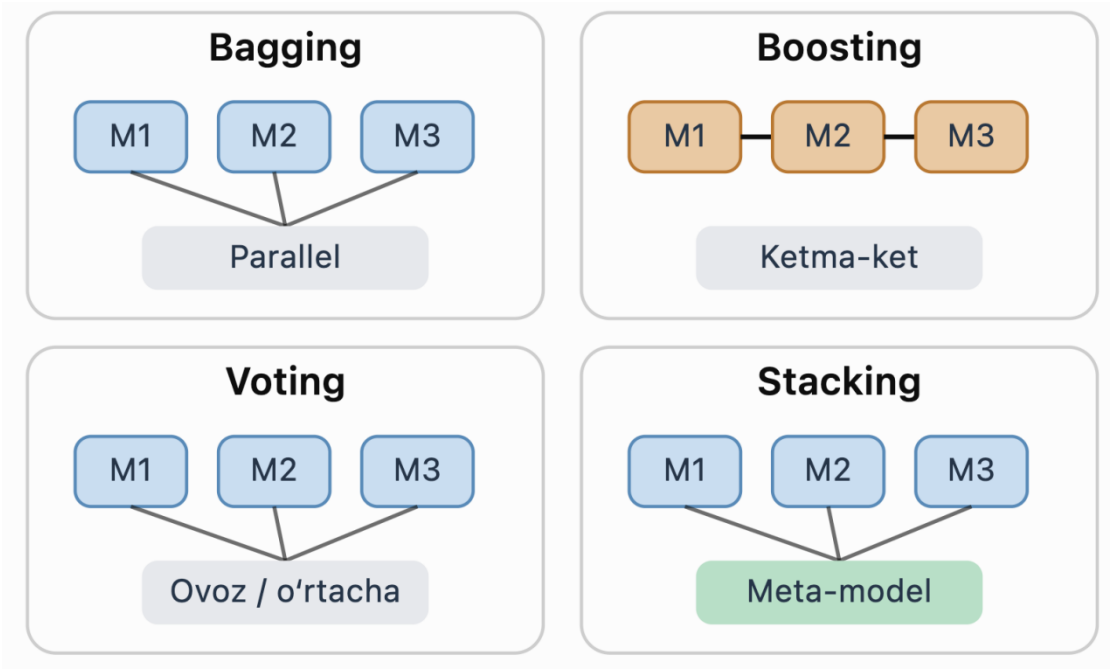
No Free Lunch Theorem (Wolpert & Macready, 1997)

Universal eng yaxshi algoritm mavjud emas. Har bir modelning kuchli va kuchsiz tomonlari dataset, data distribution va muammoga bog‘liq.

2. Ensemble Learningning 4 asosiy usuli

Method	Qanday ishlaydi?	Misollar
Bagging	Modellarni bootstrap namunalarida parallel o‘qitadi	Random Forest
Boosting	Modellarni ketma-ket o‘qitib, oldingi xatolarni kamaytiradi	AdaBoost, Gradient Boosting, XGBoost
Voting	Bir nechta model bashoratlarini ovoz yoki o‘rtacha orqali birlashtiradi	Hard / Soft Voting
Stacking	Base modellarning bashoratlaridan meta-model o‘rganadi	StackingClassifier, StackingRegressor

Ensemble usullarining ishlash sxemasi



3. Metadata va Meta-model farqi

Meta — biror narsa haqida yuqoriroq darajadagi ma'lumotni bildiradi.

Metadata = Data about Data (ma'lumot haqidagi ma'lumot).

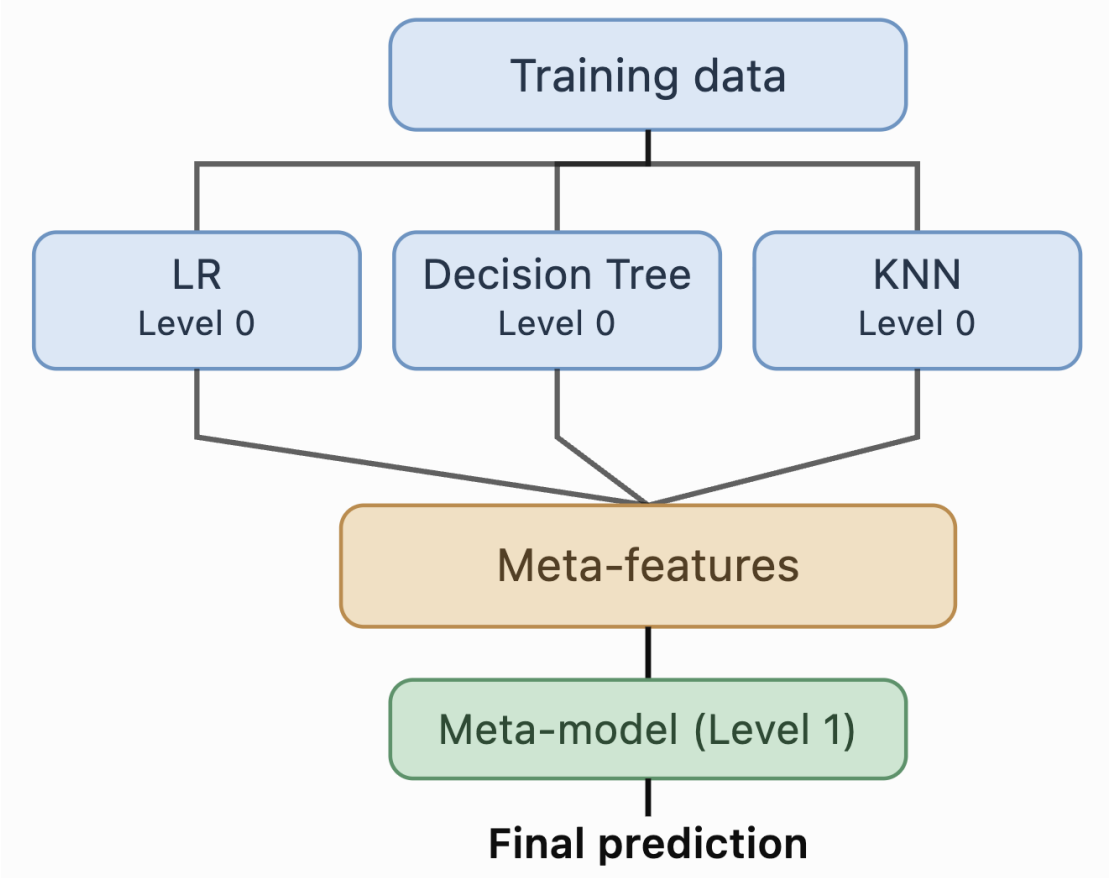
Data turi	Metadata misollari
Image file	Resolution, file size, JPEG/PNG format, created date
Dataset	Column names, data types, units, missing values count
Text document	Author, date, word count, language
Soccer dataset	Player name, column description, goals per game birligi, data source

Meta-model esa boshqa modellarning bashoratlarini input sifatida olib, yakuniy natijani o‘rganadigan modeldir.

4. Stacking (Stacked Generalization)

Stacking — bir nechta base modelning bashoratlarini yangi model yordamida birlashtirish.

Asosiy g‘oya: oddiy Voting o‘rtacha hisoblaydi, Stacking esa bashoratlarni qanday birlashtirishni o‘rganadi.



Stacking qanday ishlaydi?

1. Level 0 — Base Models: LR, Decision Tree, KNN kabi modellar o‘qitiladi.
2. Predictions → Meta-features: Har bir modelning bashorati yangi feature bo‘ladi.
3. Level 1 — Meta-model: Masalan, Ridge Regression ushbu bashoratlardan o‘rganadi.
4. Final Output: Meta-model yakuniy bashoratni chiqaradi.

Masalan:

Base model	Bashorat
Logistic Regression	0.75
Decision Tree	0.60
KNN	0.90

Meta-model inputi: [0.75, 0.60, 0.90].

Muhim: Stackingda meta-modelni o‘qitish uchun odatda Out-of-Fold (OOF) bashoratlardan foydalaniladi. Base modelning o‘z training datasidagi oddiy bashoratlarini ishlatish overfitting va leakage xavfini oshiradi.

```
from sklearn.ensemble import StackingRegressor

from sklearn.linear_model import LinearRegression
```

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score


# 1. Input data
X = df.drop("xG", axis=1)
y = df["xG"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# 2. Level 0: Base models
base_learners = [
    ("lr", LinearRegression()),
    ("dt", DecisionTreeRegressor(max_depth=5)),
    ("svr", SVR())
]
# 3. Level 1: Meta-model
meta_model = LinearRegression()
# 4. Stacking
stack = StackingRegressor(
    estimators=base_learners,
    final_estimator=meta_model,
    cv=5
)
# 5. Training
stack.fit(X_train, y_train)
```

```
# 6. Prediction
y_pred = stack.predict(X_test)
```

```
# 7. Evaluation
print("MSE:", mean_squared_error(y_test, y_pred))
print("R²:", r2_score(y_test, y_pred))
```

Kodning muhim qismlari:

1. base_learners — uchta boshlang‘ich model.
2. final_estimator — natijalarni birlashtirishni o‘rganadigan meta-model.
3. cv=5 — meta-modelni o‘qitishda besh foldli OOF bashoratlardan foydalanadi.
4. fit() — modelni o‘qitadi.
5. predict() — bashorat qiladi.
6. MSE — o‘rtacha kvadrat xato; kichik bo‘lsa yaxshi.
7. R^2 — model targetdagi o‘zgaruvchanlikni qanchalik tushuntirishini ko‘rsatadi; kattaroq bo‘lishi odatda yaxshi.

df oldindan yuklangan va barcha predictorlar raqamli, mos formatda bo‘lishi kerak.

6. Manual Stacking

```
import numpy as np


base_train_preds = []
base_test_preds = []
for name, model in base_learners:
    model.fit(X_train, y_train)
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)
    base_train_preds.append(train_pred)
    base_test_preds.append(test_pred)
    print(name)
    print("Train R²:", model.score(X_train, y_train))
    print("Test R²:", model.score(X_test, y_test))
X_meta_train = np.column_stack(base_train_preds)
X_meta_test = np.column_stack(base_test_preds)
meta_model = Ridge()
meta_model.fit(X_meta_train, y_train)
y_final_pred = meta_model.predict(X_meta_test)
```

```
print("Final predictions:", y_final_pred[:5])
print("Final R²:", r2_score(y_test, y_final_pred))
```

Bu yerda Ridge uchun from sklearn.linear_model import Ridge importi kerak.

Eslatma: Ushbu manual kod faqat Stacking mexanizmini tushuntirish uchun. U training datasidagi in-sample predictionsdan foydalanadi, shuning uchun to‘g‘ri validatsiya uchun mos emas. Amaliyotda OOF predictions yoki yuqoridagi StackingRegressor(cv=5) afzal.

Xuddi shu yondashuv classification uchun ham mavjud: StackingClassifier.

7. Stacking afzalliklari va kamchiliklari

Afzalliklar	Kamchiliklar
Accuracy yaxshilanishi mumkin	Computational cost yuqori
Bias kamayishi mumkin	Interpretability qiyinroq
Murakkab nonlinear munosabatlarni o‘rganishi mumkin	Overfitting xavfi bor
Turli modellarni birlashtiradi	Ko‘proq training va validation talab qiladi

Qachon foydalanamiz?

1. Yetarlicha katta dataset mavjud bo‘lsa.
2. Bir-birini to‘ldiruvchi, turli xatolarga ega modellar bo‘lsa.
3. Qo‘shimcha computational cost maqbul bo‘lsa.
4. Validation natijalari Stacking foyda berishini tasdiqlasa.

8. Voting vs Stacking vs Random Forest

Aspect	Voting	Stacking	Random Forest
Complexity	Low	Higher	Medium
Combination learning	No	Yes, meta-model	No, tree aggregation
Overfitting risk	Odatda pastroq	Yuqoriroq bo‘lishi mumkin	Odatda nazorat qilinadi
Performance	Ko‘pincha yaxshi	Yaxshiroq bo‘lishi mumkin	Ko‘pincha kuchli

Bagging → modellarni parallel o‘qitadi. **Boosting** → xatolarni ketma-ket kamaytiradi. **Voting** → ovoz yoki o‘rtacha orqali birlashtiradi. **Stacking** → modellar bashoratlaridan yangi model o‘rganadi.

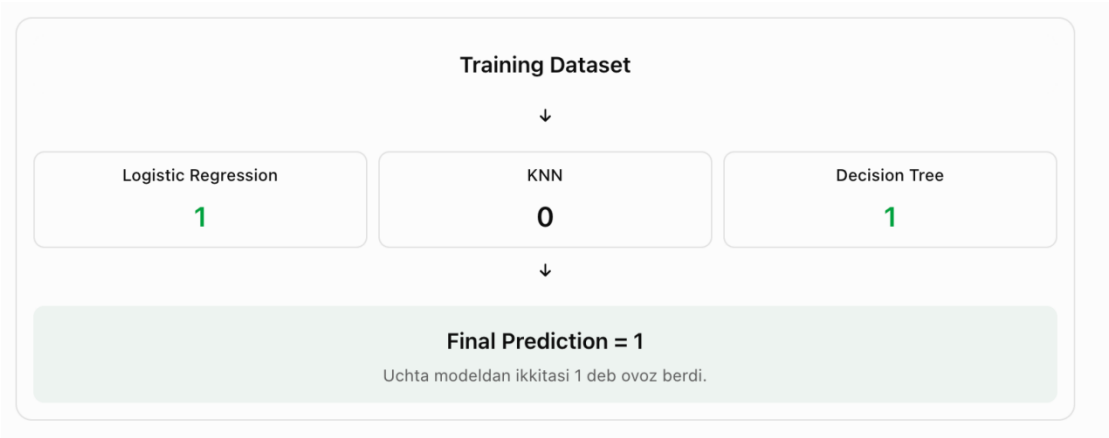
11dars:Meta Learner, Enemble Learninglarning davomi

Ensemble Learning — bir nechta Machine Learning modelini birlashtirib, yaxshiroq va barqarorroq bashorat olish usuli.

Asosiy maqsadlari: accuracy'ni oshirish, variance'ni kamaytirish, overfitting xavfini pasaytirish va turli modellarning kuchli tomonlaridan foydalanish.

1. **Base Learner:** Ensemble ichidagi alohida model.
2. **Weak Learner**: Mustaqil ishlaganda nisbatan kuchsiz model.
3. **Meta-Learner**: Boshqa modellarning bashoratlaridan o‘rganadigan model.

Voting esa bir nechta model bashoratini birlashtirishdir:



Hard Votingda ko’pchilik ovozi tanlanadi

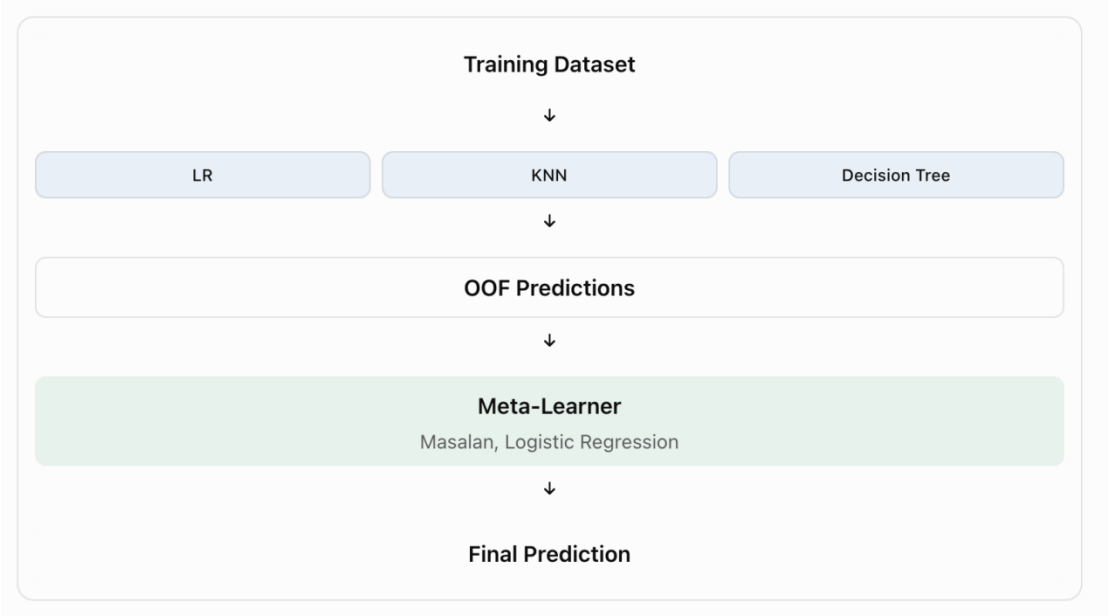
Soft Votingda esa Modellnign ehtimollari o’lchanadi.

Masalan:

$$P_{\text{final}} = \frac{0.80 + 0.60 + 0.40}{3} = 0.60$$

Threshold 0.50 bo‘lsa, prediction = 1.

Stakcing esa base modellarning bashoratlaridan fodalanib yangi meta-modelni o’qitishga aytiladi:



Muhim jihati shudnaki: meta-modelni o‘qitishda Out-of-Fold (OOF) bashoratlardan foydalanish kerak. Bu haddan tashqari optimistik natijalar va leakage xavfini kamaytiradi.

Stacking ko‘proq hisoblash vaqti talab qiladi, ammo turli modellar bir-birini to‘ldirganda foydali bo‘lishi mumkin.

Model Variance va Overfitting

Variance — training data biroz o‘zgarsa, model bashorati qanchalik o‘zgarishini ko‘rsatadi.

High Variance	Low Variance
Bashoratlar keskin o‘zgaradi	Bashoratlar barqarorroq
Overfitting xavfi yuqori bo‘lishi mumkin	Odatda barqarorroq
Data shovqiniga sezgir	Data o‘zgarishiga kamroq sezgir

Overfitting — model training datani haddan tashqari o‘rganib, yangi ma'lumotlarda yomon ishlashi.

Bagging ko‘pincha variance'ni kamaytiradi. Ammo past variance har doim yaxshi model degani emas: bias ham muhim.

Bagging — Bootstrap Aggregating

Bagging — bir xil algoritmning bir nechta nusxasini turli bootstrap sample'larda mustaqil o‘qitish va natijalarini birlashtirish.

Bootstrap Sampling: Ma'lumotlardan replacement bilan tasodifiy namuna olish.

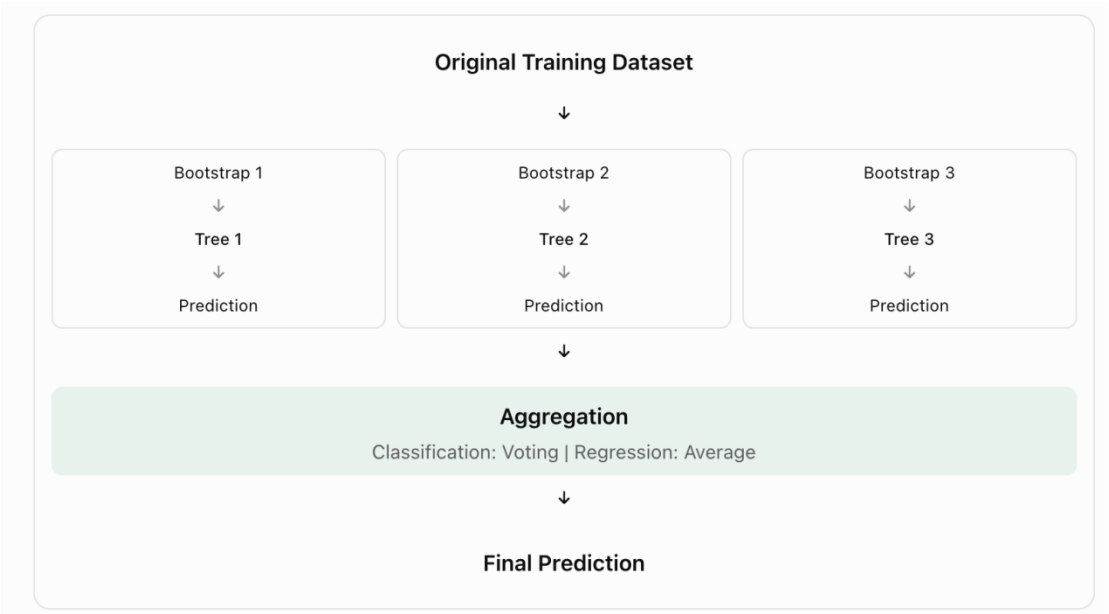
Original: [1, 2, 3]

Sample 1: [1, 1, 3]

Sample 2: [2, 1, 3]

Sample 3: [3, 3, 2]

Bir observation bir necha marta takrorlanishi mumkin.



Regression uchun formula:

$$\hat{y}(x) = \frac{1}{B}\sum_{b=1}^{B} f_b(x)$$

Bu yerda B — modellar soni, $f_b(x)$ — har bir modelning bashorati.

Bu yerda:

- $\hat{y}(x)$ — yakuniy bashorat.
- B — modellar soni.
- $f_b(x)$ — har bir modelning bashorati.
- $\sum$ — barcha prediction'larni qo‘shish.

Masalan 5 ta Decision Tree orqali modelni o’qitdik.

Bizda 5 ta model bor, demak $B = 5$.

$$\hat{y}(x) = \frac{100 + 110 + 105 + 95 + 90}{5}$$

$$= \frac{500}{5} = \boxed{100}$$

Demak, **Bagging Regression yakuniy uy narxini 100 million so‘m deb bashorat qiladi.**

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import (
    RandomForestClassifier,
    ExtraTreesClassifier,
```

```
    BaggingClassifier,
    VotingClassifier,
    StackingClassifier
)
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score

# Dataset
X = df.drop("red_cards", axis=1)
y = df["red_cards"].astype(int)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    stratify=y, random_state=42
)
# Random Forest

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Extra Trees
et = ExtraTreesClassifier(
```

```
    n_estimators=100,
    random_state=42
)
et.fit(X_train, y_train)
y_pred = et.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Bagging
bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=100,
    random_state=42
)
bag.fit(X_train, y_train)
y_pred = bag.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Voting
lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000)
)
knn = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
dt = DecisionTreeClassifier(max_depth=5)
models = [
    ("lr", lr),
    ("knn", knn),
```

```
    ("dt", dt)
]
voting = VotingClassifier(
    estimators=models,
    voting="soft"
)
voting.fit(X_train, y_train)
y_pred = voting.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))


# Stacking
stacking = StackingClassifier(
    estimators=models,
    final_estimator=LogisticRegression(),
    cv=5
)
stacking.fit(X_train, y_train)
y_pred = stacking.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

cv=5 meta-modelni o‘qitish uchun besh foldli OOF predictions hosil qiladi.

Performance Metrics

Metric	Ma'nosi
Accuracy	Barcha prediction'larning to‘g‘ri ulushi
Precision	Positive prediction'larning to‘g‘ri ulushi
Recall	Haqiqiy positive'larning topilgan ulushi
F1-score	Precision va Recall muvozanati
ROC-AUC	Klasslarni ajratish qobiliyati
AP	Precision–Recall natijasi
MSE	Regression uchun kvadrat xato
R^2	Regression modelining moslik ko‘rsatkichi

Asosiy formulalar:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2\frac{Precision \cdot Recall}{Precision + Recall}$$

Class Imbalance mavjud bo‘lsa, faqat Accuracy'ga qarab xulosa chiqarish yetarli emas.

Voting, Stacking va Bagging taqqoslash

Xususiyat	Voting	Stacking	Bagging
Base models	Odatda turli	Odatda turli	Odatda bir xil
Birlashtirish	Ovoz / ehtimol	Meta-model	Ovoz / o‘rtacha
Meta-model	Yo‘q	Bor	Yo‘q
Bootstrap	Shart emas	Shart emas	Asosiy usul
Complexity	Pastroq	Yuqoriroq	O‘rtacha

Asosiy foyda Sodda kombinatsiya Kombinatsiyani o‘rganish Variance kamaytirish

Computational Complexity

Model tanlashda faqat Accuracy emas, training time, RAM va prediction time ham hisobga olinadi.

```
import time

start = time.perf_counter()

rf.fit(X_train, y_train)

end = time.perf_counter()

print("Training time:", round(end - start, 3), "seconds")
```

Amaliy loyiha tarkibi

Faylingizdagi loyiha topshirig‘i quyidagi qismlardan iborat:

1. **Project Description** — loyiha haqida.
2. **Dataset Description** — ma'lumotlar manbasi, rows va columns.
3. **Problem Statement** — yechiladigan muammo.
4. **Prediction Goal** — target.
5. **Model Comparison** — Voting, Stacking va Bagging.
6. **Performance Evaluation** — Accuracy, Recall, F1, ROC-AUC.
7. **Computational Complexity** — training vaqti.
8. **Practical Application** — qo‘llanish sohasi.

Eng muhim eslatmalar:

Voting — modellar ovoz beradi.

Stacking — meta-model bashoratlarni birlashtirishni o‘rganadi.

Bagging — bootstrap sample'larda modellarni mustaqil o‘qitadi.

Random Forest — ko‘plab tasodifiy Decision Tree'larni birlashtiradi.

Extra Trees — split threshold'larini yanada tasodifiy tanlaydi.